

Digital Systems Project Assignment 1

Digital Watch: Settable Time

Professor Jonathan H. Manton

Across two assignments, you will design and implement a digital watch featuring settable time, a stopwatch, a timer, display brightness control, and optionally a game or two. This first assignment delivers the settable watch.

The project uses a professional development environment: VS Code with real-time linting, and a Python-based test suite that compiles and tests your modules automatically. No more waiting hours for Quartus, only to be greeted with esoteric error messages. The workflow mirrors professional practice: in industry, `iverilog` would be replaced by a commercial simulator such as Questa or VCS, but everything else remains the same.

Learning Objectives

By the end of this project, you will have:

- used a professional development environment, including real-time linting and an automated test suite;
- appreciated the value of automated tests for catching regressions and verifying behaviour, especially when refactoring;
- encountered more advanced language constructs and design patterns;
- practised two primary decomposition strategies: building many small, reusable modules, and feature layering;
- seen first-hand why parametrised designs matter for simulation and testing;
- appreciated the importance of tracking bit widths carefully;
- read and interpreted simulation waveforms to verify and debug circuit behaviour;
- seen how a synthesis tool translates an HDL description into hardware;
- run a formal verification tool to prove correctness properties of your code.

Contents

1. Policies and Assessment	3
1.1. Group Work and Authorship	3
1.2. Report	4
1.3. Final Submission	4
1.4. Backing Up Your Work	4
1.5. Oral Assessment	5
2. Assignment Summary	5
3. Environment Setup	5
4. SystemVerilog	7
5. Development Cycle	8
5.1. Module Specification	9
5.2. Write the Module	9
5.3. Lint, Compile then Generate Waveforms	11
5.4. Run the Automated Tests	14
5.5. Write Your Own Test Benches	14
5.6. Fulfil Report and Oral Preparation Obligations	15
5.7. Backup	15
6. Time Display	15
6.1. State Encoding	15
6.2. A Parametrised Up-Down Counter with Enable	16
6.3. Binary to Binary-Coded Decimal	18
6.4. Cascaded Hour-Minute-Second Counter with Enable	18
6.5. Modulo N Counter	20
6.6. Restartable Rate Generator	20
6.7. Variable-Speed Time Display	23
6.8. Programming the Board	24
7. Settable Watch	25
7.1. Fixed-Frequency Fixed-Duty PWM Generator	25
7.2. Rising-Edge Detector	25
7.3. Button Hold Detect	26
7.4. Button Hold Pulse	27
7.5. Button Auto Repeat	28
7.6. Arming Latch	29
7.7. Edit Mode Selector	29
7.8. Editable Counter	31
7.9. Key Synchroniser	32
7.10. Decimal Display Driver	33
7.11. Board Wrapper	35
7.12. Watch Specifications and Feature Layering	37
7.13. Timekeeping (Watch V1)	38
7.14. Mode Selection (Watch V2)	42
7.15. Settable Watch (V3)	43
7.16. Accurate Time Setting (V4)	43
7.17. Programming the Board	43

A. Common Pitfalls	44
A.1. Installation	44
A.2. Inexplicable Behaviour	44
A.3. VS Code	44
A.4. SystemVerilog	45
A.5. Automated Tests	45
A.6. Simulation	45
A.7. VaporView	46
A.8. Windows Filesystem Performance	46
B. SystemVerilog Initialisation	46
C. Inside the Development Environment	47
C.1. Toolchain	47
C.1.1. Installing Icarus Verilog	47
C.1.2. Using Icarus Verilog	49
C.1.3. Verilator	50
C.1.4. Universal Ctags	51
C.1.5. Python and Virtual Environments	51
C.1.6. pytest	52
C.1.7. cocotb	53
C.1.8. Why Not VUnit?	54
C.2. VS Code	55
C.2.1. Verible	56
C.2.2. SystemVerilog	56
C.2.3. Python and Ruff	57
C.2.4. VaporView	57
C.2.5. Verilog HDL	57
C.2.6. MSYS2 Terminal Profile	58
C.3. Dev Container	58
C.3.1. Configuration	59
D. Version History	62
D.1. Changes in Version 1.5	63
D.2. Changes in Version 1.4	63
D.3. Changes in Version 1.3	63
D.4. Changes in Version 1.2	63
D.5. Changes in Version 1.1	64

1. Policies and Assessment

1.1. Group Work and Authorship

Students are encouraged to work in groups; having peers with whom to discuss the project is invaluable. There is no requirement to register your group in LMS or to use the same groups as in the practical assignments.

However, **every student must maintain their own copy of the code, type their own modules, and run the automated tests independently. Even if your solution matches a group member's, type it yourself rather than copying files.** Proficiency

with the tools and familiarity with the code come only from this hands-on engagement, and both are essential at the oral.

At the start of your report, provide two lists:

- **Authors.** Everyone who wrote this report: you alone if sole-authored, or all co-authors if jointly authored.
- **Acknowledgements.** Anyone else whose input benefited this report, even informally.

1.2. Report

Boxes labelled “Report” and shown in blue, mark what to include in your report. Aim for brief, precise answers. Each Verilog module you develop also has its own report requirements, as described in Section 5; these are *not* listed where the module is introduced, and you are responsible for including them.

1.3. Final Submission

There is no weekly submission; submit once, after completing all assignments. Your submission must contain exactly two items:

1. A PDF of your report, with author and acknowledgement lists as described above.
2. A single Zip archive of your Quartus project containing your final code.

Your Zip archive must be a fully working Quartus project: a demonstrator must be able to unzip it, then compile and program the board without changing anything. Every file your project depends on must therefore live inside the project folder. `Project >> Add/Remove Files` will add a file from anywhere on disk, recording whatever path you gave it, even one outside the project folder; such a file compiles on your own computer but is silently missing from your Zip archive, and the project will then fail to compile for anyone else. **Copy every source file into the project directory (or a subdirectory of it) before adding it via `Add/Remove Files`.** If in doubt, unzip your own archive into a fresh, empty folder and confirm it compiles there before submitting.

The Quartus project should include its own author and acknowledgement lists in an `ACKNOWLEDGEMENTS.txt` file. Collaborative development is encouraged, but you must acknowledge any collaboration and independently write your own code. **One-way copying, especially without genuine discussion, is not collaboration.**

1.4. Backing Up Your Work

This is a large project. **Back up your work regularly;** losing code to hardware failure or accidental overwrite is entirely avoidable and is no excuse for failing to submit on time.

If you are familiar with version control, use **Git**: commit frequently and push to a remote repository (e.g. GitHub) after each working session.

If not, regularly copy your project folder to at least two other devices, and label each copy with a date and version.

Running automated tests regularly provides a safety net against accidental edits, for example if you bump your keyboard without noticing. Version control and manual backups then give you quick ways to undo the damage.

1.5. Oral Assessment

The oral is **individual**, regardless of how you worked. It assesses *depth of understanding*, not just what you built, so many students receive around **50%** even with a working implementation. You are expected to justify every design decision, account for all simulation waveforms, and modify any part of your code on the spot. Prepare as you develop each module, ensuring you can explain it and reproduce it from scratch.

What to Prepare All required report content is fully examinable at the oral. Boxes labelled “Additional Oral Preparation” and shown in brown, flag further topics that are equally examinable. Each module you develop has its own oral preparation requirements, as described in Section 5; these are *not* listed where the module is developed.

2. Assignment Summary

Think of this project document as **written material in lieu of a textbook**. Learning by doing is the most productive way to learn, and only by doing do you begin to appreciate the nuances that separate novices from experts; this document highlights those nuances as they arise.

Much of that learning can happen anywhere: the virtual environment simulates everything you need, so no hardware is required until you are ready to test on the board. By the end of this first assignment, you will have a settable digital watch, ready to program onto your board and capable of keeping accurate time. Lab time is best used to discuss your work with demonstrators and peers, test on the board, and see how others approach the project.

To help you navigate what follows, the table of contents provides a clear overview of the assignment, with embedded hyperlinks to jump straight to any section. In short: you will first be guided through a development cycle, then apply it to each new module in turn, and finally reflect on the development process. For every module, you must also fulfil the report and oral preparation requirements described in Section 5.

3. Environment Setup

Although hardware and software engineering mindsets are quite different, software-configurable hardware means much of modern hardware development now uses the same tools and best practices as software development. You will see this first-hand in this project.

Virtual environments solve the common challenge of giving everyone on a team the same development environment, even when they use different computers. Without them, “It works on my computer” is a phrase you hear all too often.

You will obtain the virtual environment (and the automated test suite written for this project) in one of two ways, with details provided on LMS. You can either clone a Git

repository or download a zip file. In both cases, you will end up with your **project directory**, where you will do your work.

- Download and install Docker Desktop.
 - If you are on Windows, install WSL beforehand, as Docker requires it: right-click the Start button or press `Win+X`, select `Terminal (Admin)`, `PowerShell (Admin)`, or `Command Prompt (Admin)` (whichever appears), then run `wsl --install`.
 - If WSL fails to work, check that virtualisation is enabled in the BIOS; on some machines, you may also need to download a firmware update from your manufacturer's website.
- Download and install Visual Studio Code (VS Code).
- Install Git (optional but recommended). If unsure how, VS Code will offer to install it: open `View >> Source Control`.
- Run Docker Desktop; you can minimise it once it starts up. (The Dev Containers extension, once installed, will also try to launch Docker Desktop automatically if it is not already running.)
- Run VS Code.
- In VS Code, go to `View >> Extensions` and search for Dev Containers by Microsoft. Install it.
- You may also wish to install the `vscode-pdf` extension by tomoki1207 to view PDF files directly inside VS Code.
- If you are using `git`, clone the repository in VS Code. One way is to press `Ctrl+Shift+P` to open the command palette, then run `Git: Clone` and follow the prompts.
- If you have a zip file, unzip it, then in VS Code go to `File >> Open Folder` and select the folder that *directly* contains the `.devcontainer` directory. **This is often one level below the top-level unzipped folder; opening the wrong folder means the container prompt will not appear.**
- VS Code should offer to reopen the folder in a container (usually as a notification in the bottom-right corner); click `Reopen in Container`. If the notification does not appear, press `Ctrl+Shift+P`, type `Dev Containers: Reopen in Container`, and select it.
- Another message will appear indicating progress.
 - Packages will be downloaded from the Ubuntu repository; this may take a few minutes depending on your network speed.
- Quite soon, you will be in your virtual environment and have access to tools specifically designed for Verilog development.
- If inexplicable behaviour occurs, first verify **Verilog (ELEN20006)** appears in the lower-left blue label, then **rebuild the dev container** (see Appendix A.2).

Becoming proficient with VS Code is a skill that will serve you well beyond this subject, not only for development but also for report writing in LaTeX. The same workflows — editing, navigation, linting, and version control — will transfer directly to later subjects and professional work.

4. SystemVerilog

For several reasons, this project uses SystemVerilog, a superset of Verilog. This should make your work easier in two key ways.

- SystemVerilog has stronger linting support, allowing tools to catch more potential mistakes.
- SystemVerilog offers convenient features that are unavailable in Verilog.

When writing code, you only need to make a few changes.

Use `.sv` instead of `.v` as the file extension.

Use `always_ff @(posedge clk)` instead of `always @(posedge clk)`. This makes it clear to both the reader and the tools that the block describes flip-flops.

Use `always_comb` instead of `always @(*)`. This makes it clear that the block describes combinational logic and turns buried implied-latch warnings into explicit errors.

You should use `logic` instead of `wire` or `reg`, but note one important difference.

```
1 wire x = y;    // Continuous assignment
2 logic x = y;   // Only assigned at time 0 !!!
```

Instead, state your intent explicitly:

```
1 logic x;
2 assign x = y;
```

Other convenient SystemVerilog features will be introduced gradually throughout the project, as you first need them.

Wire, Reg and Logic A `wire` is a net: it represents a physical connection and must be driven continuously, either by an `assign` statement or by module instantiation. A `reg` is a variable: it is assigned procedurally within an `always` or `initial` block and holds its value until the next assignment.

In SystemVerilog, `logic` is a variable type that can stand in for both `wire` and `reg` in almost all cases. The notable exception is that `logic z = t;` makes a one-time assignment at time 0, not a continuous one (see Appendix A.4). In practice, you can use `logic` throughout and not worry about `wire` or `reg`.

Port Declarations We recommend declaring all ports as `logic`. In other code you may see the type omitted for ports that would be `wire` in Verilog (ports default to `wire`); any port that would have been `reg` still requires `logic`.

Recommendation Use `logic` everywhere, and avoid declarations such as `logic x = y;`.

Initialisation SystemVerilog has changed initialisation behaviour; Appendix B elaborates. The practical consequence for this project is that `initial` blocks should not appear in synthesisable code. Use immediate initialisation instead: `output logic [3:0] count = '0;` in a port declaration, or `logic [3:0] count = '0;` in the module body.

5. Development Cycle

The golden rule of software development — which applies equally to Verilog development — is to write a *small* amount of code, compile and test it thoroughly, finalise its documentation, and then leave it alone unless you need to fix a bug down the track. We use this approach to build small, reusable modules. Near the end, when these modules must be assembled, we shift to a feature-layered strategy: add a small feature, compile and test thoroughly, then move on to the next feature.

Thousands of lines of automated test code have been written for you. In real life, you or your team would be responsible for developing and maintaining this test code. As the saying goes, “*if it is not tested, it is broken.*” Test code is critical: it improves reliability, prevents costly defects, makes changes safer, and serves as living documentation.

Each write-compile-test loop follows what we call the *development cycle*. This cycle is exemplified below.

Conventions A good project has an agreed set of conventions.

Each module should live in its own file within the `rtl/` subdirectory, with the filename matching the module name. Names should be descriptive and should only be abbreviated when the full name would be overly long. Well-established abbreviations such as `clk` and `rst` are acceptable, provided they are used consistently.

Naming conventions are as follows. (Commonly, `localparam` constants are also capitalised, but our particular linter disagrees.)

Kind	Convention	Example
Modules and signals	<code>snake_case</code>	<code>next_state</code> , <code>tick_enable</code>
<code>parameter</code> constants	<code>ALL_CAPS</code>	<code>MAX</code> , <code>WIDTH</code>
<code>localparam</code> constants	<code>CamelCase</code>	<code>TicksPerSecond</code>

File Locations The files for each module are organised across four subdirectories.

rtl/ SystemVerilog source files. Each module has its own file here.

tb/ Waveform test benches. Each module has a matching test bench that compiles with the module and generates a `.vcd` file.

tests/ Pytest entry points. Each file parametrises test runs over the relevant parameter values and calls the corresponding cocotb test module for each configuration.

tests/cocotb/ Cocotb test modules. Each file contains test logic for one parameter configuration.

5.1. Module Specification

For each module, you will be given its declaration together with a description of the required behaviour. Follow the specification exactly, or the automated tests will fail.

Below is an example specification, for a module called `seven_segment`; Section 5.2 explains what to do with such a specification.

```
1 module seven_segment #(
2     parameter int ACTIVE_LOW = 1
3 ) (
4     input logic [3:0] digit,
5     input logic      blank,
6     output logic [6:0] segments
7 );
```

When `blank` is high, all segments must be off, regardless of `digit`. Otherwise, `segments` must encode the standard seven-segment representation of `digit` (`4'h0–4'hF`), with bit 0 corresponding to segment `a` and bit 6 to segment `g`: `a` is top, `b` top-right, `c` bottom-right, `d` bottom, `e` bottom-left, `f` top-left, and `g` middle.

When `ACTIVE_LOW == 0`, 1 turns a segment on; when `ACTIVE_LOW == 1`, 0 turns a segment on.

Notes SystemVerilog provides the keyword `int`, whereas Verilog uses `integer`. The difference is that `integer` can take on “unknown” and “high-impedance” values, whereas `int` is a genuine numeric type. Use `int`.

Declaring every port as `logic` conveys the correct semantics — each port carries a logic signal — and simplifies matters: there is no need to decide between `wire` and `reg`.

5.2. Write the Module

Whenever you are asked to write a new module — starting now, for the `seven_segment` specification above — do the following:

1. Create a file with the matching filename and extension `.sv` in the `rtl` subdirectory.
2. At the top of the file, include ``timescale 1ns / 1ps`.
3. Copy the module declaration from the specification.

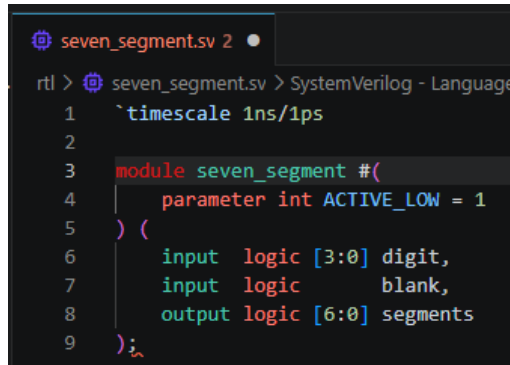
If you miss any one of these steps, the automated tests or test benches will fail.

Six VS Code tips:

- Use the **View** menu to open the **Explorer**, **Terminal**, and **Testing** panels.
- To create a new file, right-click the target directory in the **Explorer** panel and select **New File**.
- To format your file, right-click inside the editor and select **Format Document**. This requires the file to be free of errors; if it silently does nothing, see Appendix A.3.
- To save, press **Ctrl** + **S**; VS Code will format your file automatically on save.

- Near the top-right of the **Explorer** panel is the **Collapse Folders** icon; click it periodically to keep the file tree tidy.
- With multiple files open, drag their tabs to create a split view; useful when instantiating modules.

You should now be seeing the following. (If not, delete lines 10 and beyond.)



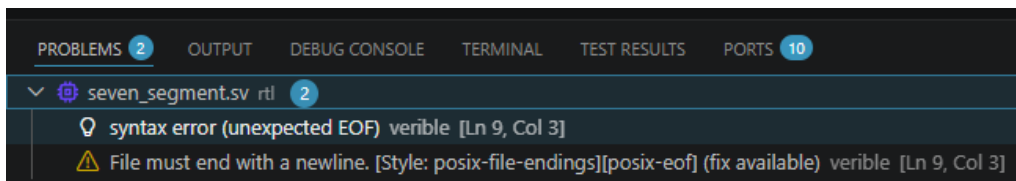
```

seven_segment.sv 2
rtl > seven_segment.sv > SystemVerilog - Language
1 `timescale 1ns/1ps
2
3 module seven_segment #(
4     parameter int ACTIVE_LOW = 1
5 ) (
6     input logic [3:0] digit,,
7     input logic      blank,,
8     output logic [6:0] segments
9 );

```

Pay attention to the visual clues from the linter. The tab shows the filename in red, followed by the number 2. Red indicates an error, and 2 indicates two total issues (errors and warnings combined). A red squiggle marks the error location on line 9. Hovering over it shows “syntax error (unexpected EOF)”. The keyword `module` also appears in red (from syntax highlighting rather than the linter, but still a useful cue). If you try to format the document at this point, nothing happens.

Clicking **PROBLEMS** in the bottom panel displays the full list of issues; if the bottom panel is not visible, select **View** > **Terminal** first.



PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS 10

seven_segment.sv rtl 2

- 🔴 syntax error (unexpected EOF) variable [Ln 9, Col 3]
- 🟡 File must end with a newline. [Style: posix-file-endings][posix-eof] (fix available) variable [Ln 9, Col 3]

Add `endmodule` on line 10, but do not press **Enter**. The filename colour changes from red to yellow, indicating that warnings remain, but errors do not. A yellow squiggle then appears at the end of `endmodule`, indicating that the file must end with a newline; you can confirm this by hovering over it or by checking the **PROBLEMS** view in the bottom panel. You do not need to fix this manually: format the document or save it. Either action formats the document and adds a newline at the end. The tab should then show `seven_segment.sv` in white, with no number beside it.

Because there is no simple algorithmic mapping from a number to the corresponding LED configuration on a seven-segment display, we need a `case` statement. A suitable block has been prepared for you. Open the `templates` directory, right-click `seven_segment_case_block.txt`, and choose **Open to the Side**. This opens the file in a second editor panel. In that panel, press **Ctrl**+**A**, then **Ctrl**+**C** to copy everything. Return to `seven_segment.sv`, make room for the block, and paste it in (**Ctrl**+**V**).

We now have errors again. A `case` statement cannot appear on its own. Because we are describing combinational logic, it must sit inside an `always` block. SystemVerilog makes

this explicit: add `always_comb` before `case`. The errors should disappear, and you can then format the file by saving it (`Ctrl`+`S`).

Hovering over the yellow squiggle on `case` reveals: “Explicitly define a default case for every case statement or add ‘unique’ qualifier to the case statement.” In SystemVerilog, `unique case` means exactly one branch should match at run time, and tools can warn if none or multiple branches match. Here, since all hexadecimal inputs are covered, using `unique case` is appropriate and avoids a redundant `default` branch.

Add `unique` before `case`. The linter should then report no warnings or errors. Save your file.

The `case` statement we inserted assigns to `segments_active_high`, so this signal must be declared. In SystemVerilog, we do not need to decide between `wire` and `reg` here. Add `logic segments_active_high;` before the `always_comb` block, then save. We have intentionally not yet given `segments_active_high` the correct width.

One final line completes the design:

```
1 assign segments = ACTIVE_LOW ? !segments_active_high : segments_active_high;
```

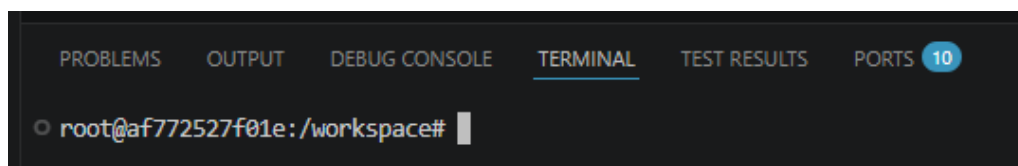
There is another intentional mistake here, but because the linter does not complain, we save and proceed to the next step.

Add Comments It is good practice to summarise your module’s behaviour at the top of the file. For example:

```
1 // Seven-segment display decoder for hexadecimal digits.
2 //
3 // Parameters:
4 //   ACTIVE_LOW    - 1 for active-low LEDs, 0 for active-high.
5 //
6 // Ports:
7 //   digit    [3:0] - Hexadecimal digit to display (0x0 to 0xF).
8 //   blank    - When high, all segments are turned off.
9 //   segments [6:0] - Segment outputs [g,f,e,d,c,b,a].
```

5.3. Lint, Compile then Generate Waveforms

Open a terminal via `View` `Terminal`. If the panel shows the dev container build log rather than a shell prompt, use `Terminal` `New Terminal` instead. Your prompt should resemble the image below, possibly preceded by a line activating the Python virtual environment and a `(venv)` prefix on the prompt; this is normal.



If you are familiar with Unix, you will feel right at home: this is a `bash` shell. One particularly useful feature is Tab auto-completion. Type only part of a filename, then press `Tab`; for example, type `ls rtl/s`, press `Tab`, then press `Enter` to run the command.

With auto-completion, if pressing `Tab` does nothing, press it again. If this shows a list of possibilities, type one or more letters, then press `Tab` again.

Pressing `↑` recalls previous commands, which you can rerun by pressing `Enter`.

Your current directory is shown in the prompt and should be `/workspace`; if not, run `cd /workspace` to return there. The project is set up so that you should never need to change directory.

In this assignment we use the terminal for four things (in the next assignment, it will also be used to run a formal verification tool):

1. More thorough linting (`verilator -Wall --lint-only`).
2. Compiling code (`iverilog -Wall -g2012`).
3. Running test benches (`iverilog -Wall -g2012`, then `./a.out`).
4. Running the automated test suite (`pytest`).

In these commands, `-Wall` enables all warnings, and `-g2012` enables SystemVerilog support. Although `iverilog` does not yet provide full SystemVerilog support, it is more than adequate for our purposes.

Lint Run: `verilator -Wall --lint-only rtl/seven_segment.sv`

It reports 20 warnings before stopping. These are warnings rather than errors: the code is valid SystemVerilog and will compile. **Do not mistake “warning” for “harmless”: a warning usually means the tool has noticed you doing something you did not intend.** Most of the warnings below correspond to genuine bugs. Even the one that does not, noted as it arises, should still be eliminated: being explicit about your intent is good practice, and the goal is always zero warnings.

Sixteen warnings relate to the case items: a 7-bit value is being assigned to a 1-bit variable. Fix this by declaring `logic [6:0] segments_active_high`; with the correct width, then rerun `verilator` (`↑`, then `Enter`).

Three of the remaining warnings relate to the `assign` statement: the bit widths are incorrect. We used logical NOT instead of bitwise NOT. Fix this, then rerun Verilator.

Of the two remaining warnings, the first reports that `ACTIVE_LOW` is 32 bits wide, whereas a logical expression should be 1 bit. Although the code is valid, the goal is zero warnings, and this issue is easy to fix: replace `ACTIVE_LOW` with the more explicit `ACTIVE_LOW != 0`.

This leaves just one warning: we forgot about `blank`. Insert a suitable `if` statement before the `unique case` statement to resolve it.

Report 1

Explain why Verilator accepts `if (blank)` but not `if (ACTIVE_LOW)`.

Compile Once linting reports zero warnings, compile the module and its corresponding test bench.

```
iverilog -Wall -g2012 rtl/seven_segment.sv tb/wave_seven_segment.v
```

The file extensions differ because we write modules in SystemVerilog (.sv) while the test benches are written in Verilog (.v).

You will see an apology message:

```
root@af772527f01e:/workspace# iverilog -Wall -g2012 rtl/seven_segment.sv tb/
    wave_seven_segment.v
rtl/seven_segment.sv:16: vvp.tgt sorry: Case unique/unique0 qualities are ignored.
```

The message indicates iverilog will not enforce **unique** at compile time. This is not a problem: verilator checks **unique** for us, and the simulator still checks at runtime.

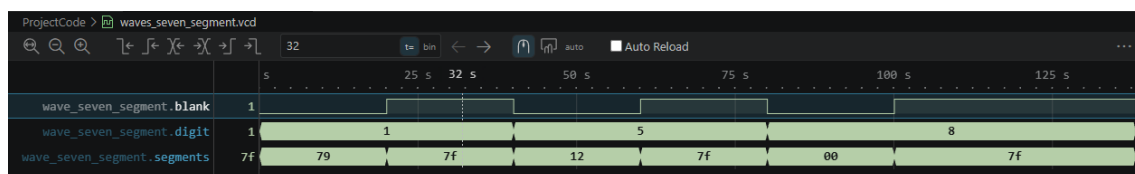
Simulate The iverilog command produces the executable a.out; running ./a.out creates wave_seven_segment.vcd containing the waveforms.

```
$ iverilog -Wall -g2012 rtl/seven_segment.v tb/wave_seven_segment.v
$ ./a.out
VCD info: dumpfile wave_seven_segment.vcd opened for output.
tb/wave_seven_segment.v:40: $finish called at 120 (1s)
```

View Waveforms In the VS Code Explorer, click the .vcd file to open it in VaporView. If the Explorer is not visible, open it via [View](#) [Explorer](#).

Click the VaporView icon in the Activity Bar to open the Netlist Panel. Double-click a signal to add it to the waveform display. To change the display format (binary, hexadecimal, decimal, and so on), right-click the signal and choose [Format Values](#). For details on navigation, markers, and other features, consult the VaporView documentation via the Extensions panel.

The waveform should resemble the following:



Signal values are shown in hexadecimal. When blank is high, the output is 7f, which corresponds to 1111111 in binary — correct for an active-low display. The waveforms confirm the expected behaviour.

Notes A VaporView bug can block certain VS Code operations, such as saving a file in another window. Close the VaporView window immediately after use.

If you have installed a wave viewer outside of VS Code, you may use that instead; your project files are accessible from your computer as normal.

5.4. Run the Automated Tests

From the terminal, run `pytest tests/test_seven_segment.py` to test only this module, or `pytest` to test all modules. Tests for unwritten modules will be skipped.

Check that all tests for `tests/test_seven_segment.py` pass. If they are reported as skipped, the file is either misnamed or not in the `rtl` subdirectory.

If any tests fail, work through the following steps:

- Scroll up through the `pytest` output to identify which test failed.
- Read the comments in `tests/cocotb/tb_seven_segment.py` to understand what the failing test checks.
- Focus on the corresponding part of your module.

If you are still stuck, see the next section.

Notes

- See Appendix C.1.6 for useful `pytest` options.
- Automated tests are invaluable after refactoring, when small mistakes can easily go unnoticed.
- If you find a bug not caught by the tests, notify the test bench author so that a check can be added.
- Once you are familiar with `pytest`, you may prefer the GUI. Open the **TESTING** view by clicking the flask icon in the Activity Bar or selecting **View** **Testing**. Then hover over **Project workspace**, move the mouse slightly to the right, and click the first Play button.

5.5. Write Your Own Test Benches

If your module is still not working, write your own test bench. Although the error dump from `pytest` is verbose, and much of it may be unintelligible, scroll through it for messages that make sense. There will usually be a comment explaining what the test was checking, followed by a statement of why it failed. Target that same behaviour in your test bench: your aim is to reproduce the fault.

If reproducing the faulty behaviour requires many steps, simplify it to the fewest steps that still exhibit the fault. Focus on one bug at a time.

Once you have a test bench that reproduces the bug, trace your code through the same steps. This should reveal where your code behaves unexpectedly.

The other side of the coin is defensive programming. As this project emphasises, a module should be both small *and* simple: if the code cannot be understood in seconds rather than minutes, decompose it into smaller modules. When further decomposition is not feasible, adopt a feature-layering approach and add code gradually, one feature at a time, for the same reason.

Using small, focused modules is best practice because it keeps debugging manageable: a small module has fewer possible sources of error, and its input and state combinations

grow much more slowly, which helps you avoid a combinatorial explosion in what you need to reason about and test.

5.6. Fulfil Report and Oral Preparation Obligations

The following obligations apply to every module, including `seven_segment`.

Report 2

For every module, include a screenshot of the waveforms and briefly explain why they are correct, pointing to specific features in the traces.

Additional Oral Preparation 1

For every module, be able to describe its behaviour precisely for all input combinations.

Additional Oral Preparation 2

Be able to explain the development cycle and the project conventions in detail.

5.7. Backup

After each module passes all its tests, back up your code and report.

Report 3

At the start of your report, describe your backup strategy.

6. Time Display

By the end of this section, the seven-segment displays will show hours, minutes, and seconds, incrementing once per second like a clock. Switches select faster rates. Time is in 24-hour format.

6.1. State Encoding

Before writing any code, we must decide how to represent the current time in hardware. There are at least four reasonable choices.

1. **One counter (clock cycles).** Count the total number of rising edges since midnight. With a 50 MHz clock, a rising edge occurs every 20 ns.
2. **One counter (seconds).** Count the total number of seconds elapsed since midnight.
3. **Three counters.** Maintain separate counters for hours (0–23), minutes (0–59), and seconds (0–59).
4. **Six counters.** Maintain a separate counter for each display digit: the tens and units of hours, minutes, and seconds.

The choice of state encoding directly influences the number of flip-flops required, the complexity of the next-state logic, and the complexity of the output logic.

The encoding choice also influences the time-setting hardware developed later. Time-setting works by cycling through each unit (seconds, minutes, and hours) and letting the user increment or decrement each independently.

This project adopts the three-counter encoding: separate counters for hours, minutes, and seconds.

Additional Oral Preparation 3

Be able to explain all four state encoding options and the trade-offs between them, and justify why the three-counter encoding was chosen.

6.2. A Parametrised Up-Down Counter with Enable

You must follow the development cycle for every module (Section 5). Among other things, this includes eliminating all Verilator lint warnings and errors, generating and including waveforms in your report, and ensuring that all automated tests pass.

```
1 module up_down_counter #(
2     parameter int MAX    = 2,
3     parameter int WIDTH = 2
4 ) (
5     input logic clk,
6     input logic enable,
7     input logic up,
8     output logic [WIDTH-1:0] count
9 );
```

The count is initialised to 0.

When `enable` is low, count does not change.

When `enable` is high, count increments if `up` is high and decrements if `up` is low, wrapping from MAX to 0 on increment and from 0 to MAX on decrement.

Rationale To edit time conveniently, we must be able to increment and decrement the count.

Implementation Notes It is conventional¹ to gate `enable` inside the `always_ff` block, even though it could instead be absorbed into the next-state logic.

```
1 always_ff @(posedge clk) if (enable) count <= next_count;
```

Use `always_comb` for the next-state logic.

Verilator will flag bit-width mismatches; use SystemVerilog's cast notation to adjust the width of a value explicitly.

¹This mirrors real flip-flops, many of which have a dedicated clock-enable pin.

```

1 // Triggers a linter warning: 0 is 32 bits wide.
2 output logic [WIDTH-1:0] count = 0
3
4 // Width set explicitly
5 output logic [WIDTH-1:0] count = WIDTH'(0)

```

SystemVerilog also provides '0, '1, 'x and 'z, which replicate that value to match the required width. **In particular, '1 is all ones.** An expression like '2 is a syntax error.

```

1 // Correct
2 output logic [WIDTH-1:0] count = '0;
3
4 // Incorrect
5 assign next_count = count + '1;

```

To satisfy the linter, cast MAX into a correctly-sized local parameter.

```

1 localparam logic [WIDTH-1:0] Max = WIDTH'(MAX);
2
3 // Linter warning
4 if (count < MAX) ...
5
6 // Correct
7 if (count < Max) ...

```

When instantiating a parametrised module, however, still use MAX rather than Max.

```

1 counter #(.MAX(MAX)) u_counter(...);

```

Quartus is stricter about widths than Verilator. To suppress truncation warnings, you have two options.

```

1 // Option 1
2 assign next_count = count + WIDTH'(1);
3
4 // Option 2
5 localparam logic [WIDTH-1:0] One = WIDTH'(1);
6 assign next_count = count + One;

```

Quartus is not without leniency, however: the following truncates a 32-bit number without issuing a warning, as the intent is considered unambiguous.

```

1 localparam logic [WIDTH-1:0] One = 1;

```

6.3. Binary to Binary-Coded Decimal

```
1 module binary_to_bcd (  
2     input logic [6:0] bin,    // binary input, 0-99  
3     output logic [3:0] tens,  // decimal tens digit (BCD)  
4     output logic [3:0] ones   // decimal ones digit (BCD)  
5 );
```

For any input value `bin` in the range 0 to 99, the output `tens` represents the decimal tens digit of `bin`, and the output `ones` represents the decimal ones digit.

Rationale Displaying hours, minutes and seconds on seven-segment displays requires a binary to binary-coded decimal converter.

Report 4

Explain what binary-coded decimal (BCD) is. You may need to research it.

Implementation Notes The linter expects the same bit width on both sides of a division. Since `bin` is 7 bits wide, we must write `bin / 7'd10`.

Writing `assign tens = bin / 7'd10` will still produce a warning: the LHS (left-hand side) is 4 bits wide, but the RHS (right-hand side) is 7 bits wide. As we genuinely want to discard the leading bits, we must explicitly convert the result to 4 bits.

```
1 assign tens = 4'(bin / 7'd10);
```

This may seem pedantic when you want to finish the assignment quickly, but in mission-critical designs, you must always be conscious of bit width. The linter forces you to be explicit about your intent.

When viewing the waveforms, set the viewer format for `bin` to decimal.

6.4. Cascaded Hour-Minute-Second Counter with Enable

```
1 module hms_counter #(  
2     parameter int N_HOURS   = 24, // number of hours  
3     parameter int N_MINUTES = 60, // number of minutes  
4     parameter int N_SECONDS = 60, // number of seconds  
5  
6     // Output port widths  
7     parameter int W_HOURS   = 5,  
8     parameter int W_MINUTES = 6,  
9     parameter int W_SECONDS = 6  
10 ) (  
11     input logic clk,  
12     input logic enable,  
13     output logic [W_HOURS-1:0] hours,  
14     output logic [W_MINUTES-1:0] minutes,
```

```
15     output logic [W_SECONDS-1:0] seconds
16 );
```

When `enable` is low, all outputs retain their values.

When `enable` is high, `seconds` increments on each rising clock edge. On a seconds rollover (when `seconds` is at `N_SECONDS-1` and `enable` is high), `seconds` wraps to zero and `minutes` increments. Likewise, on a minutes rollover, `minutes` wraps to zero and `hours` increments; on an hours rollover, `hours` wraps to zero.

Rationale We introduced parameters for testing, while deliberately keeping the module name specific rather than generic. Smaller parameter values do more than speed up simulation: they make bugs easier to find, since the test bench is shorter and debugging is more manageable. Had we intended this module to be reused as a general cascaded counter, we would have given it a more generic name instead. Because its name is specific, it is fine to instantiate `hms_counter` without overriding any parameters: the defaults already implement exactly what the name promises.

Implementation Notes Follow these conventions in the implementation:

- Declare two `logic` variables named `second_rollover` and `minute_rollover`.
- Instantiate three `up_down_counter` modules called `u_hour`, `u_minute`, and `u_second`. The `u_` prefix, short for unit, marks these as module instances and avoids name clashes with signals.

To eliminate linter warnings, we must use width-adjusted parameter versions. The following contains a subtle bug:

```
1 // Incorrect!
2 localparam logic [W_MINUTES-1:0] NMinutes = W_MINUTES'(N_MINUTES);
3 localparam logic [W_SECONDS-1:0] NSeconds = W_SECONDS'(N_SECONDS);
```

The widths passed to the module are suitable for storing numbers from 0 to $N - 1$, but the code above stores N using the same number of bits. In edge cases, such as $N = 2$, it stores 0 instead of 2 due to truncation that you explicitly authorised.

Because the code needs $N - 1$ rather than N , the solution is straightforward:

```
1 localparam logic [W_MINUTES-1:0] MaxMinutes = W_MINUTES'(N_MINUTES - 1);
2 localparam logic [W_SECONDS-1:0] MaxSeconds = W_SECONDS'(N_SECONDS - 1);
```

Careful bit-width tracking is vital: writing a logic condition like `x == y+1` is dangerous because `y+1` might wrap around to zero. This danger exists in other programming languages too, but it is most apparent in hardware design, where we work with small numbers of bits. The extra effort to be precise with widths costs minutes and can save hours of debugging.

For linting and compilation, include all required files.

```
verilator -Wall --lint-only rtl/hms_counter.sv rtl/up_down_counter.sv
iverilog -Wall -g2012 rtl/hms_counter.sv rtl/up_down_counter.sv tb/
        wave_hms_counter.v
```

Use the `Tab` key for autocompletion to save typing.

6.5. Modulo N Counter

```
1 module mod_n_counter #(
2     parameter int N = 4,
3     parameter int WIDTH = 2
4 ) (
5     input logic clk,
6     input logic rst,
7     input logic enable,
8     output logic [WIDTH-1:0] count
9 );
```

The output `count` is initially zero.

On a rising clock edge, `rst` takes priority over `enable`: a high `rst` clears `count` regardless of `enable`.

If only `enable` is high, `count` advances. When it reaches $N - 1$, the next enabled edge wraps it back to zero; otherwise it increments by one.

Rationale Even if `up_down_counter` had a reset pin, a dedicated module with a minimal interface makes the design intent clearer than one with several pins tied high or low.

Implementation Notes Refer to your previous modules for the techniques needed to suppress linter warnings.

It is customary² to place reset logic inside the flip-flop block rather than in the next-state logic. Use `always_ff @(posedge clk)` rather than `always @(posedge clk)`; the linter will not flag the latter, but `always_ff` makes the intent explicit.

```
1 always_ff @(posedge clk)
2     if (rst) count <= '0;
3     else if (enable) count <= next_count;
```

6.6. Restartable Rate Generator

```
1 module restartable_rate_generator #(
2     parameter int CYCLE_COUNT = 2
3 ) (
4     input logic clk,
5     input logic run,
6     output logic tick
7 );
```

²This mirrors real flip-flops, many of which have a dedicated reset pin.

When `run` is low, `tick` goes low.

When `run` is high:

- After `run` has been high for exactly `CYCLE_COUNT-1` rising edges, `tick` is high for exactly one clock cycle.
- Thereafter, after each period of `run` having remained high for `CYCLE_COUNT` rising edges, `tick` is again high for exactly one clock cycle.

The edge case `CYCLE_COUNT = 1` is supported: `tick` follows `run` directly.

The module is to be implemented as a Moore FSM.

Rationale This module drives the watch’s one-second timekeeping and is reused during time editing, where holding a button causes repeated increments or decrements. Supporting `CYCLE_COUNT = 1` ensures the design remains functional when the watch is configured to tick every clock cycle, which is useful for debugging.

There is no functional reason for this module to be a Moore FSM; the requirement is imposed purely as a learning opportunity.

Implementation Notes There is no elegant way to write this module directly: it is just complex enough that a direct implementation becomes awkward. Instantiating a `mod_n_counter` avoids this, keeping the higher-level logic uncluttered.

In a Moore FSM, outputs depend only on state. Since `tick` must be low whenever `run` is sampled low, `run` must be captured as part of the state.

For `CYCLE_COUNT > 1`, the remaining state is a counter with modulus `CYCLE_COUNT`. Its width is an internal concern — it does not affect any port — and must be computed internally rather than imposed on the instantiating module.

```
1 localparam int CountWidth = $clog2(CYCLE_COUNT);
```

Report 5

Explain how `localparam CounterWidth = $clog2(CYCLE_COUNT);` works.

When `CYCLE_COUNT = 1`, `CounterWidth` is zero, so no counter is needed; a `generate` statement handles this by conditionally omitting the instantiation. Blocks inside a `generate` statement must be named — here `g_general` and `g_special` — or the linter will warn.

```
1 generate
2   if (CYCLE_COUNT > 1) begin : g_general
3       // Put code for the general case here,
4       // including counter instantiation
5   end else begin : g_special
6       // Put code for the special case here
7   end
8 endgenerate
```

With readability in mind, we start with the high-level structure:

```

1 // Becomes high at the end of each cycle
2 logic tick_qualifier;
3
4 logic running = 1'b0;
5 always_ff @(posedge clk) running <= run;
6
7 assign tick = running && tick_qualifier;

```

It remains for the **generate** block to provide `tick_qualifier`. The special case is straightforward:

```

1 begin : g_special
2     assign tick_qualifier = 1'b1;
3 end

```

A template for the general case follows, with the logic kept separate from the module instantiation.

```

1 begin : g_general
2     localparam int CountWidth = $clog2(CYCLE_COUNT);
3
4     logic rst_count;
5     logic enable_count;
6     logic [CountWidth-1:0] count;
7     mod_n_counter #(
8         .N(CYCLE_COUNT),
9         .WIDTH(CountWidth)
10    ) u_count (
11        .clk(clk),
12        .rst(rst_count),
13        .enable(enable_count),
14        .count(count)
15    );
16
17    assign rst_count = ...; // Fill in
18    assign enable_count = ...; // Fill in
19
20    assign tick_qualifier = ...; // Fill in
21 end

```

To ensure the correct bit width, use `CountWidth'(CYCLE_COUNT - 1)` when needed.

Report 6

In addition to running the provided test bench, write your own test bench verifying correct behaviour when `CYCLE_COUNT = 1`.

Note The module's behaviour may appear asymmetric: the initial delay is `CYCLE_COUNT-1` rising edges, whereas subsequent ticks repeat every `CYCLE_COUNT` rising edges. However, if `run` is synchronised to the clock (i.e., it transitions just after a rising edge, as it would if driven from a flip-flop), the initial delay is also `CYCLE_COUNT` clock cycles, matching the repeat rate.

6.7. Variable-Speed Time Display

```
1 module top_time_display_v1 #(
2     parameter int CYCLES_PER_SECOND = 50_000_000
3 ) (
4     input logic CLOCK_50,
5     input logic [1:0] SW,
6     output logic [6:0] HEX5,
7     output logic [6:0] HEX4,
8     output logic [6:0] HEX3,
9     output logic [6:0] HEX2,
10    output logic [6:0] HEX1,
11    output logic [6:0] HEX0
12 );
```

This top-level module for the DE1-SoC board displays the time on the seven-segment displays, initialised to 00:00:00, with the tick rate controlled by SW[1:0].

SW[1:0]	Tick Rate
2'b00	1 Hz
2'b01	25 Hz
2'b10	1 kHz
2'b11	50 MHz

Rationale The goal is to have a working design ready for the DE1-SoC board when you arrive at the lab. Designing and debugging in VS Code beforehand means it should work first time.

Implementation Notes The final code may be around 100 lines but is straightforward. Begin by instantiating:

- one `hms_counter`;
- three `restartable_rate_generators`, one for each of the slower tick rates;
- three `binary_to_bcds`, one each for hours, minutes and seconds; and
- six `seven_segments`, one for each of the digits.

If it is not immediately obvious what to connect to a port, create a named `logic` variable for it and assign it a value using an `assign` statement at the bottom of the module. This keeps instantiation separate from logic and improves readability, provided the variables are named clearly.

The outputs of `hms_counter` are narrower than the `bin` input of `binary_to_bcd`. Explicitly zero-extend them when making each connection, for example `.bin({2'b0, hours})`.

For each `restartable_rate_generator`, use `CYCLES_PER_SECOND` to derive the `CYCLE_COUNT` value. The default of `50_000_000` matches the DE1-SoC's 50 MHz clock; the test suite overrides it to a smaller value to keep simulation fast.

As the design grows, we can save time by telling `verilator` or `iverilog` to search `rtl/` for any modules it needs. For `iverilog`, we also specify `.sv` as the file extension to search. Note that `-Y .sv` must come before `-y rtl`.

```
verilator -Wall --lint-only -y rtl rtl/top_time_display_v1.sv
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_top_time_display_v1.v
```

The automated tests check the 1 kHz tick rate but not the slower rates (25 Hz and 1 Hz); these can be verified by hand once the design is on the board.

The simulator may produce warnings labelled `Time: 0`, such as the following.

```
root@a374924768bc:/workspace# ./a.out
VCD info: dumpfile wave_top_time_display_v1.vcd opened for output.
WARNING: rtl/seven_segment.sv:30: value is unhandled for priority or
        unique case statement
        Time: 0 Scope: wave_top_time_display_v1.dut.u_HEX0
```

This is due to a race condition: the `unique case` statement in `seven_segment` must have executed before at least one of the `up_down_counter` instances has initialised its output, so some or all bits of `digit` are unknown. Since no case item matches an unknown value, the `unique` constraint is correctly reported as violated. There are three solutions.

1. Ignore `unique case` warnings labelled `Time: 0`.
2. Add a `$isunknown(digit)` check to handle unknown inputs explicitly.
3. Some simulators allow warnings at time 0 to be suppressed.

If these warnings truly bother you, add the following to `seven_segment`:

```
1 if (blank || $isunknown(digit)) segments_active_high = '0;
```

Quartus will ignore this construct, and since the warnings are a simulator quirk, adding extra logic to suppress them is not ideal. The most pragmatic solution is simply to ignore `unique case` warnings labelled `Time: 0`.

6.8. Programming the Board

Skip to the next section if you are not in the lab; return here when you are ready to program the board.

Follow these steps to program the DE1-SoC:

1. Create a new Quartus project (e.g., `digital_watch`) with the correct settings for the DE1-SoC board.
2. Set the top-level module to `top_time_display_v1`.
3. Import assignments from `DE1_SoC.qsf`.
4. Create an `.SDC` file specifying the clock period.
5. Copy all your `rtl/*.sv` files into the Quartus project root directory (or under `rtl/` if you prefer).
6. Go to Project » Add/Remove Files and add these files to the project.
7. Compile and program the board.

7. Settable Watch

By the end of this section, we will have a settable watch: time displayed on the DE1-SoC board, incrementing once per second, with the ability to edit the time using the keys. Holding KEY[3] for at least one second enters edit mode, with the seconds digits flashing at 2 Hz with an 80% duty cycle. A second press of KEY[3] moves to the minutes digits, a third to the hours digits, and a fourth exits edit mode.

In edit mode, KEY[1] increments and KEY[0] decrements the flashing digits. Holding either key for more than half a second repeats the increment or decrement at 10 Hz.

7.1. Fixed-Frequency Fixed-Duty PWM Generator

```
1 module pwm_generator #(
2     // Number of clock cycles in one PWM period
3     parameter int PERIOD_CYCLES = 50_000_000,
4
5     // Number of clock cycles output is high
6     parameter int DUTY_CYCLES = 25_000_000
7 ) (
8     input logic clk,
9     input logic rst,
10    output logic pwm_out
11 );
```

Report 7

Explain why this module is called a PWM generator.

Rationale During edit mode, the display flashes to indicate which field is being edited. This module provides the PWM signal that drives the flashing.

Implementation Notes Instantiate a `mod_n_counter`. To see its port names while working, right-click `mod_n_counter.sv` in the Explorer panel and select *Open to the Side*.

Report 8

In `mod_n_counter`, `rst` takes priority over `enable`. Comment on the benefit of this design choice.

7.2. Rising-Edge Detector

```
1 module rising_edge_detector (
2     input logic clk,
3     input logic sig_in,
```

```
4     output logic rise
5 );
```

The output `rise` is asserted immediately when `sig_in` transitions from low to high, and deasserted as soon as `sig_in` returns low or the next rising clock edge captures the high value, whichever comes first.

Rationale At 50 MHz, a button press spans millions of clock cycles. Without edge detection, a single press of the edit button could trigger millions of mode transitions. A rising edge detector converts the press into a single-cycle pulse, ensuring exactly one transition per press.

Unlike the other modules in this assignment, `rising_edge_detector` is a Mealy FSM — the industry-standard choice for edge detection — as it avoids a one-cycle output delay, is simpler to write, and produces output logic simple enough to be glitch-free.

7.3. Button Hold Detect

```
1 module button_hold_detect #(
2     parameter int HOLD_CYCLES = 50_000_000
3 ) (
4     input logic clk,
5     input logic button,
6     output logic held
7 );
```

The output `held` is asserted on the clock edge at which `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges, and deasserts on the clock edge after `button` goes low.

Rationale Entering edit mode requires the user to hold a button for a minimum duration; this module detects when that threshold has been reached.

Implementation Notes Instantiate a `mod_n_counter` and think carefully about its maximum count. The counter value is the entire FSM state; no additional state variables are needed.

It is more natural to reason about this counter in terms of its maximum value than its modulus N . Define `CountMax` explicitly to make the intent clear and avoid off-by-one errors.

```
1 localparam int CountMax = ... ; // Fill this in
2 localparam int CountWidth = $clog2(CountMax + 1);
3
4 logic count_rst;
5 logic count_enable;
6 logic [CountWidth-1:0] count;
7 mod_n_counter #(
8     .N(CountMax + 1),
9     .WIDTH(CountWidth)
```

```

10 ) u_counter (
11     .clk(clk),
12     .rst(count_rst),
13     .enable(count_enable),
14     .count(count)
15 );

```

From here, derive the next-state and output logic. You may find it convenient for the next-state logic to depend on the output `held`; this is permissible.

Report 9

Explain why the next-state logic may depend on the output `held` without violating the Moore FSM constraint.

Use `CountWidth'(CountMax)` where needed to ensure correct bit widths.

Note As in Section 6.6, the hold duration is exactly `HOLD_CYCLES` clock cycles only if the button is synchronised to the clock.

7.4. Button Hold Pulse

This module is provided in full; nevertheless, go through the full development cycle: run `verilator`, `iverilog`, and the automated tests, and include the waveforms in your report.

```

1 module button_hold_pulse #(
2     parameter int HOLD_CYCLES = 50_000_000
3 ) (
4     input logic clk,
5     input logic button,
6     output logic pulse
7 );
8
9     logic held;
10
11     button_hold_detect #(
12         .HOLD_CYCLES(HOLD_CYCLES)
13     ) u_detect (
14         .clk(clk),
15         .button(button),
16         .held(held)
17     );
18
19     rising_edge_detector u_detector (
20         .clk(clk),
21         .sig_in(held),
22         .rise(pulse)
23     );
24
25 endmodule

```

Report 10

Explain what this module does. Justify your explanation by referring to the generated waveforms, being precise about timing.

Report 11

Explain how `button_hold_pulse` is a Moore FSM despite `rising_edge_detector` being Mealy.

Note Because `rising_edge_detector` is a Mealy FSM, `pulse` asserts on the same clock edge as `held`, rather than one cycle later. A Moore implementation could achieve the same timing by passing `HOLD_CYCLES - 1` to `button_hold_detect`.

7.5. Button Auto Repeat

```
1 module button_auto_repeat #(
2     parameter int HOLD_CYCLES    = 50_000_000,
3     // REPEAT_CYCLES must be smaller than HOLD_CYCLES
4     parameter int REPEAT_CYCLES = 5_000_000
5 ) (
6     input logic clk,
7     input logic button,
8     output logic pulse
9 );
10
11 logic rise;
12 logic held;
13 logic pulse_train;
14
15 assign pulse = rise | (button & pulse_train);
```

The two internal signals, `rise` and `pulse_train`, are derived as follows.

- `button` → `rising_edge_detector` → `rise`
- `button` → `button_hold_detect` → `restartable_rate_generator` → `pulse_train`

A brief press produces an immediate pulse; holding the button produces a pulse train.

The first pulse of the pulse train occurs after `button` has been sampled high for `HOLD_CYCLES` consecutive rising edges; subsequent pulses repeat every `REPEAT_CYCLES` rising edges.

Rationale When editing the watch, a brief press should advance the time by one step; holding the button should produce rapid repeated advances. This module provides both behaviours.

Implementation Notes Instantiate three modules:

- `rising_edge_detector`
- `button_hold_detect`
- `restartable_rate_generator`

Choose their parameters carefully.

Note The rising edge detector is Mealy, so the overall design is Mealy. There is therefore no reason to avoid gating `pulse_train` with `button`: using `button & pulse_train` eliminates the possibility of a spurious pulse one clock cycle after the button is released.

7.6. Arming Latch

```
1 module arming_latch (  
2     input  logic clk,  
3     input  logic arm,  
4     input  logic disarm,  
5     output logic armed  
6 );
```

Initially `armed` is low. On each rising clock edge, if `disarm` is high, `armed` is cleared; if only `arm` is high, `armed` is set.

Rationale One instance of this module will track whether the watch is in edit mode.

Note This is simply a flip-flop with synchronous set and clear, where clear takes priority; the port names reflect intended use rather than implementation.

7.7. Edit Mode Selector

```
1 module edit_mode_selector #(  
2     parameter int HOLD_CYCLES = 50_000_000  
3 ) (  
4     input logic clk,  
5     input logic button,  
6     output logic [2:0] mode_enable  
7 );  
8  
9     logic long_press;  
10    button_hold_pulse #(  
11        .HOLD_CYCLES(HOLD_CYCLES)  
12    ) u_hold_pulse (  
13        .clk(clk),  
14        .button(button),  
15        .pulse(long_press)  
16    );  
17  
18    logic press;  
19    rising_edge_detector u_detector (  
20        .clk(clk),  
21        .sig_in(button),  
22        .rise(press)  
23    );  
24  
25    logic armed;
```

```

26  logic disarm;
27  arming_latch u_latch (
28      .clk(clk),
29      .arm(long_press),
30      .disarm(disarm),
31      .armed(armed)
32  );
33
34  logic reset_counter;
35  logic enable_counter;
36  logic [1:0] count;
37  mod_n_counter #(
38      .N(3),
39      .WIDTH(2)
40  ) u_mod_3_counter (
41      .clk(clk),
42      .rst(reset_counter),
43      .enable(enable_counter),
44      .count(count)
45  );
46
47  // Counter runs only while armed; resets when disarmed
48  assign enable_counter = ... ;    // Fill this in
49  assign reset_counter = ... ;    // Fill this in
50
51  // Disarm on the press that steps past the last mode
52  assign disarm = ... ;           // Fill this in
53
54  // Output logic
55  assign mode_enable = armed ? (3'b001 << count) : 3'b000;
56
57  endmodule

```

When count is zero, button must be held long enough to assert `long_press`, which arms the latch and allows count to advance to one. Thereafter, each brief press of `button` advances count by one until it wraps back to zero and the cycle repeats.

Rationale One instance of this module will track which field of the watch is being edited, with `mode_enable` encoding the current state:

mode_enable	Meaning
3'b000	No field selected
3'b001	Edit seconds
3'b010	Edit minutes
3'b100	Edit hours

Implementation Notes Start with a block diagram of the circuit, then derive the combinational logic for `enable_counter`, `reset_counter`, and `disarm`.

Report 12

Include your block diagram and explain the combinational logic you used for `enable_counter`, `reset_counter`, and `disarm`.

Additional Oral Preparation 4

Be able to explain the output logic.

Read these notes only after completing your design.

- Rather than asserting `reset_counter` only on the disarming transition, it is safer and simpler to hold it high continuously whenever `armed` is low.
- As a further safety measure, `enable_counter` should also be low whenever `armed` is low. This is redundant if `reset_counter` is driven correctly, but it makes the design intent explicit.
- To derive `disarm`, avoid detecting the 2-to-0 transition in `count` directly. Instead, identify the condition under which `u_mod_3_counter` increments `count`, then AND that condition with `count == 2'd2`.
- If you incorporated `reset_counter` into that condition, convince yourself that it is redundant.

Note While a single bespoke FSM would work, good hardware design favours composing behaviour from smaller, reusable components. Doing so frees you from reasoning about the design as a monolithic state diagram; instead, you can think in terms of signal flow: when one signal asserts, a few clock cycles later another asserts, which in turn causes a third to respond. Each component encapsulates one behaviour, and wiring them together produces a design that is easier to reason about, test, and reuse.

7.8. Editable Counter

```
1 module editable_counter #(
2     parameter int N = 60,
3     parameter int WIDTH = 6
4 ) (
5     input logic clk,
6     input logic tick, // Count increments on tick when edit_mode is low
7     input logic edit_mode,
8     input logic inc, // Count increments by one when edit_mode is high
9     input logic dec, // Count decrements by one when edit_mode is high
10    output logic [WIDTH-1:0] count
11 );
12
13 logic enable;
14 logic up;
15 up_down_counter #(
16     .MAX (N - 1),
17     .WIDTH(WIDTH)
18 ) u_counter (
19     .clk(clk),
20     .enable(enable),
21     .up(up),
22     .count(count)
23 );
24
25 wire inc_event = edit_mode && inc && !dec;
26 wire dec_event = edit_mode && dec && !inc;
```

```

27     wire tick_event = !edit_mode && tick;
28
29     assign up = ... ;           // Fill this in
30     assign enable = ... ;      // Fill this in
31 endmodule
32

```

This module wraps an up/down counter with two modes of operation. In normal mode, `count` increments on each `tick` pulse. In edit mode, asserting `inc` increments `count` and asserting `dec` decrements it; if both are asserted simultaneously, `count` does not change.

Rationale Three instances of this module will hold the hours, minutes, and seconds of the watch.

Implementation Notes Assign `up` and `enable` using only the three event signals. If both `inc` and `dec` are high simultaneously, `count` should not change.

Report 13

Explain why `wire` was used instead of `logic` for the three event signals.

7.9. Key Synchroniser

```

1 module key_synchroniser (
2     input logic      clk,
3     input logic [3:0] key_n,    // active-low, asynchronous
4     output logic [3:0] key_sync // active-high, synchronised
5 );

```

This module operates bitwise: each input bit is inverted then synchronised through a two-stage flip-flop cascade, with both stages initialised to zero.

Rationale The DE1-SoC pushbuttons are active-low, so they must be inverted before use. Digital inputs generally also require debouncing and synchronisation; since the DE1-SoC pushbuttons are debounced in hardware, we need only synchronise them.

Without synchronisation, a signal that transitions near a rising clock edge can leave a flip-flop in a metastable state (neither a clean 0 nor a clean 1). Passing the signal through two flip-flops in series mitigates this: even if the first flip-flop enters metastability, it has a full clock cycle to resolve to a clean 0 or 1 before the second samples it, which it will with vanishingly high probability. Crucially, it does not matter which way the first flip-flop resolves: because `key_n` is a single-bit signal, the only consequence is that the button transition may be detected one cycle late. This is far less serious than metastability in a multi-bit signal such as an FSM state register, where the resolved value matters and the system could jump to a completely wrong state.

Implementation Notes Pass each bit of `key_n` through an inverter and then two flip-flops in series to yield the corresponding bit of `key_sync`. Use bitwise operations to avoid repeating the logic four times, and initialise both flip-flops to zero explicitly.

Note Clear naming conventions reduce the risk of polarity errors: `key_n` is active-low, while `key_sync` is active-high, as indicated by the absence of the `_n` suffix.

7.10. Decimal Display Driver

Copy the following template from `templates/decimal_display_driver.sv` to your `rtl/` folder and edit it.

```
1 // -----
2 // decimal_display_driver
3 // -----
4 // Board-specific display driver for the DE1-SoC HEX displays.
5 //
6 // This module presents three decimal values (0-99) on the six
7 // seven-segment HEX displays (HEX5..HEX0).
8 // Each value uses two digits.
9 //
10 // The module accepts numeric values and blanking controls.
11 // Internally, it handles:
12 //   - binary-to-BCD conversion
13 //   - seven-segment decoding
14 //   - digit blanking
15 //   - active-low segment polarity
16 // -----
17 `timescale 1ns / 1ps
18
19 module decimal_display_driver (
20     // Decimal values to be displayed (range: 0-99)
21     // value0 -> HEX1 (tens) and HEX0 (ones)
22     // value1 -> HEX3 (tens) and HEX2 (ones)
23     // value2 -> HEX5 (tens) and HEX4 (ones)
24     input logic [6:0] value0,
25     input logic [6:0] value1,
26     input logic [6:0] value2,
27
28     // Blanking controls for each decimal value.
29     // When asserted, both digits of the value are blanked.
30     input logic blank0,
31     input logic blank1,
32     input logic blank2,
33
34     // DE1-SoC seven-segment display outputs.
35     // Active-low segments: [g,f,e,d,c,b,a]
36     output logic [6:0] HEX0,
37     output logic [6:0] HEX1,
38     output logic [6:0] HEX2,
39     output logic [6:0] HEX3,
40     output logic [6:0] HEX4,
41     output logic [6:0] HEX5
42 );
43
44 // Pack scalar ports into arrays for use in the generate loop.
45 // value_a[i] drives HEX(2i+1) (tens) and HEX(2i) (ones).
46 logic [6:0] value_a[3];
47 assign value_a[0] = value0;
48 assign value_a[1] = value1;
```

```

49     assign value_a[2] = value2;
50
51     logic blank_a[3];
52     assign blank_a[0] = blank0;
53     assign blank_a[1] = blank1;
54     assign blank_a[2] = blank2;
55
56     logic [6:0] hex_a[6];
57     assign HEX0 = hex_a[0];
58     assign HEX1 = hex_a[1];
59     assign HEX2 = hex_a[2];
60     assign HEX3 = hex_a[3];
61     assign HEX4 = hex_a[4];
62     assign HEX5 = hex_a[5];
63
64     genvar i;
65     generate
66         for (i = 0; i < 3; i = i + 1) begin : g_digit_pair
67             // Fill this in
68         end
69     endgenerate
70
71 endmodule

```

This module accepts three 7-bit decimal values (`value0`–`value2`, range 0–99) and three blanking controls (`blank0`–`blank2`). Each value is displayed on a pair of seven-segment displays: `value0` drives `HEX1` (tens) and `HEX0` (ones), and so on up to `value2` on `HEX5` and `HEX4`. When a blanking control is asserted, both digits of the corresponding value are blanked. The six outputs are active-low.

Rationale This module allows the rest of the design to treat the display as three independent decimal values, hiding the binary-to-BCD conversion, seven-segment decoding, digit blanking, and active-low polarity.

Implementation Notes A `generate` loop is a compile-time construct: the synthesiser unrolls it and produces one copy of the hardware per iteration, just as if you had written the block out three times by hand. Inside the loop body you can declare local wires and instantiate modules, using `i` as an array index to connect each copy to the correct ports. The `begin`–`end` block must be named so that local wires in each iteration occupy distinct namespaces and do not clash with those in other iterations.

The template has already packed the scalar ports into arrays and set up the `generate` loop; your task is simply to fill in the body of the named block. Declare any necessary local wires, then instantiate `binary_to_bcd` and `seven_segment`, using `i` to index the correct array elements.

Notes The automated tests do not cover every possible input. Since this module composes `binary_to_bcd` and `seven_segment`, both of which are tested separately, only a few tests are included to catch basic wiring errors.

Although 15 lines are needed to pack the ports into arrays, each line is straightforward to verify by eye. Repeating the circuit three times by hand would be harder to check carefully, and reader fatigue would make errors easier to miss.

7.11. Board Wrapper

Copy `templates/user_top.sv` and `templates/top_de1_soc.sv` to your `rtl/` directory. Their contents are provided below for reference.

```
1 // -----
2 // WARNING: This file is used by the automated test suite. Do not
3 // modify it.
4 //
5 // This file also serves as a template for your own designs. To use
6 // it:
7 //   1. Copy the entire contents into a new file with a descriptive
8 //      name.
9 //   2. Delete the test logic below and replace it with your own
10 //      code.
11 //   3. In top_de1_soc, change the module name from user_top to your
12 //      new module name.
13 //
14 //   The board wrapper sets CYCLES_PER_SECOND; use this parameter in
15 //   your design wherever timing is needed.
16 // -----
17 `timescale 1ns / 1ps
18
19 module user_top #(
20     /* verilator lint_off UNUSEDPARAM */
21     parameter int CYCLES_PER_SECOND = 50_000_000
22     /* verilator lint_on UNUSEDPARAM */
23 ) (
24     input logic clk,
25     input logic [3:0] button,
26     input logic [9:0] sw,
27     output logic [9:0] led,
28     output logic [6:0] hours_disp,
29     output logic [6:0] minutes_disp,
30     output logic [6:0] seconds_disp,
31     output logic blank_hours,
32     output logic blank_minutes,
33     output logic blank_seconds
34 );
35
36 assign led = clk ? sw : ~sw;
37
38 assign blank_hours = button[0];
39 assign blank_minutes = button[1];
40 assign blank_seconds = button[2];
41
42 assign hours_disp = button[3] ? 7'd16 : 7'd7;
43 assign minutes_disp = button[3] ? 7'd38 : 7'd23;
44 assign seconds_disp = button[3] ? 7'd59 : 7'd45;
45
46 endmodule
```

```
1 // -----
2 // Board wrapper for the DE1-SoC.
3 //
```

```

4  // This module handles all board-specific concerns:
5  //   - Synchronises the KEY inputs and converts them to active-high.
6  //   - Drives the HEX displays from decimal values.
7  //   - Sets CYCLES_PER_SECOND to the DE1-SoC clock frequency
8  //     (50 MHz).
9  //
10 // To load a different design, instantiate it in place of user_top
11 // below.
12 // -----
13 `timescale 1ns / 1ps
14
15 module top_de1_soc (
16     input logic CLOCK_50,
17     input logic [3:0] KEY, // active-low, asynchronous
18     input logic [9:0] SW,
19     output logic [9:0] LEDR,
20     output logic [6:0] HEX0,
21     output logic [6:0] HEX1,
22     output logic [6:0] HEX2,
23     output logic [6:0] HEX3,
24     output logic [6:0] HEX4,
25     output logic [6:0] HEX5
26 );
27
28     logic [3:0] button;
29
30     key_synchroniser u_key_synchroniser (
31         .clk      (CLOCK_50),
32         .key_n     (KEY),
33         .key_sync(button)
34     );
35
36     logic [6:0] hours, minutes, seconds;
37     logic blank_hours, blank_minutes, blank_seconds;
38
39     // Replace user_top with your user top-level module
40     user_top #(
41         .CYCLES_PER_SECOND(50_000_000)
42     ) u_user_top (
43         .clk      (CLOCK_50),
44         .button    (button),
45         .sw        (SW),
46         .led       (LEDR),
47         .hours_disp (hours),
48         .minutes_disp (minutes),
49         .seconds_disp (seconds),
50         .blank_hours (blank_hours),
51         .blank_minutes(blank_minutes),
52         .blank_seconds(blank_seconds)
53     );
54
55     decimal_display_driver u_decimal_display_driver (
56         .value0(seconds),
57         .value1(minutes),
58         .value2(hours),
59         .blank0(blank_seconds),
60         .blank1(blank_minutes),
61         .blank2(blank_hours),

```

```

62         .HEX0  (HEX0),
63         .HEX1  (HEX1),
64         .HEX2  (HEX2),
65         .HEX3  (HEX3),
66         .HEX4  (HEX4),
67         .HEX5  (HEX5)
68     );
69
70 endmodule

```

There are no waveforms to generate for this task; go straight to running the automated tests to verify you have copied the files correctly. There is no separate test for `user_top`: it contains the code that enables `top_de1_soc` to be tested, so it is covered by that test.

Rationale Two engineering principles worth following are *separation of concerns* and *don't repeat yourself*. Applied to our watch design, they suggest creating a dedicated *user* top-level module whose ports are meaningful to the design rather than to any particular board.

A second module, the *board wrapper*, then serves purely as the synthesis top-level that Quartus targets. It maps the board's physical pins to the user top-level's ports and sets board-specific parameters such as the clock frequency. Whenever the design is ported to a different board, only the board wrapper needs to change.

Additional Oral Preparation 5

Be able to explain the purpose of a board wrapper and how it is used.

Note Do not modify these files directly; the automated tests depend on them remaining unaltered. To adapt them for a different design, copy both files, rename the copies, and update the module names.

7.12. Watch Specifications and Feature Layering

With all the building blocks in place, we can now implement the user top-level module for the settable watch. The full specification is below.

Normal Mode

- Time increments once per second.
- Holding `KEY[3]` for ≥ 1 s enters edit mode with the seconds field selected.

Edit Mode The selected digits flash at 2 Hz with an 80 % duty cycle.

KEY[3] Advance the selection: seconds $\rightarrow$ minutes $\rightarrow$ hours $\rightarrow$ exit.

KEY[1] Increment the selected field.

KEY[0] Decrement the selected field.

Hold KEY[1] or KEY[0] for > 0.5 s Repeat at 10 Hz.

Fields below the selected field continue to tick normally. The selected field and all fields above it do not advance while in edit mode.

Before reading on, take a few minutes to draw a block diagram of this design.

You may have found the design difficult: it does not decompose naturally into submodules, because the interdependencies make it hard to identify clean boundaries. This is typical of top-level modules with many interrelated features.

Implementing everything at once is risky: a large body of code accumulates before any of it can be tested, and the more untested code there is, the harder bugs are to locate.

The solution is *feature layering*: begin with the most basic functionality, verify it works, then add features one at a time, testing and debugging each before moving on.

In the following sections, you will implement `user_top_watch` with a version suffix (`_v1`, `_v2`, and so forth); the corresponding test bench files follow the same convention. The automated tests are cumulative: each version must pass all prior tests as well as its own.

Linting and Simulation Commands Do not use wildcards when linting or simulating (see Appendix A.6). **Use the flags below**, which instruct the tools to locate modules in `rtl/`.

```
verilator -Wall --lint-only -y rtl rtl/user_top_watch_v1.sv
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v1.v
```

7.13. Timekeeping (Watch V1)

The base layer is a reimplementaion of the time display from Section 6, using three `editable_counter` instances in place of `hms_counter`.

Feature layering requires a different approach from writing independent modules. This section walks through the process in detail; **you are then expected to apply these practices independently**.

In the VS Code Explorer, right-click `rtl/user_top.sv` and select Copy, then right-click the `rtl` folder and select Paste. Rename the resulting file to `user_top_watch_v1.sv`, open it, and change the module name from `user_top` to `user_top_watch_v1`. Remove the test code inside, then add a comment just after the module declaration labelling the block as the basic time display.

```
1 // -----
2 // Core Functionality
3 // -----
```

We start from the outputs and work backwards: the hours, minutes, and seconds outputs are driven by `editable_counter` instances, so we add the seconds instance first, declaring new **logic** signals for each connection with no driver yet (there is an intentional error below):

```

1  // Seconds
2  logic seconds_tick;
3  logic seconds_edit;
4  logic seconds_inc;
5  logic seconds_dec;
6  editable_counter #(
7      .N(60),
8      .WIDTH(6)
9  ) u_seconds (
10     .clk(clk),
11     .tick(seconds_tick),
12     .edit_mode(seconds_edit),
13     .inc(seconds_inc),
14     .dec(seconds_dec),
15     .count(seconds_disp)
16 );

```

We need the same for minutes and hours. A `generate` block is not appropriate here, as naming each field explicitly keeps the code clear. Instead, use VS Code's multi-cursor: copy the seconds block, place the cursor on the first occurrence of `seconds` in the copy, press `Ctrl+D` to select each subsequent occurrence, type the replacement, press `Escape` to exit multi-cursor mode, then repeat for hours.

```

1  // Minutes
2  logic minutes_tick;
3  logic minutes_edit;
4  logic minutes_inc;
5  logic minutes_dec;
6  editable_counter #(
7      .N(60),
8      .WIDTH(6)
9  ) u_minutes (
10     .clk(clk),
11     .tick(minutes_tick),
12     .edit_mode(minutes_edit),
13     .inc(minutes_inc),
14     .dec(minutes_dec),
15     .count(minutes_disp)
16 );
17
18 // Hours
19 logic hours_tick;
20 logic hours_edit;
21 logic hours_inc;
22 logic hours_dec;
23 editable_counter #(
24     .N(24),
25     .WIDTH(5)
26 ) u_hours (
27     .clk(clk),
28     .tick(hours_tick),
29     .edit_mode(hours_edit),
30     .inc(hours_inc),
31     .dec(hours_dec),
32     .count(hours_disp)

```

33);

Conceptually, we have installed three chips and wired up their `clk` and `count` pins, deliberately leaving the rest dangling so we can tackle them one by one. We start with `seconds_tick`, which should be driven by a `restartable_rate_generator` parameterised by `CYCLES_PER_SECOND`.

```
1 // Derive 1 Hz tick from system clock
2 restartable_rate_generator #(
3     .CYCLE_COUNT(CYCLES_PER_SECOND)
4 ) u_divider_1_Hz (
5     .clk (clk),
6     .run (1'b1),
7     .tick(seconds_tick)
8 );
```

Next is `seconds_edit`. Since this version has no edit functionality, we tie it low.

```
1 assign seconds_edit = 1'b0;
```

Do the same for `minutes_edit` and `hours_edit`, then tie `inc` and `dec` low across all three instances (six signals in total).

That leaves `minutes_tick` and `hours_tick`: when should each counter increment?

```
1 assign minutes_tick = ... ; // Fill in the suitable condition
2 assign hours_tick = ... ; // Fill in the suitable condition
```

We might think we are done at this point. Let us lint and see, using `-y rtl` and listing only the top-level module file.

```
root@a374924768bc:/workspace# verilator --Wall --lint-only -y rtl rtl/
user_top_watch_v1.sv
%Warning-WIDTH: rtl/user_top_watch_v1.sv:42:8: Output port connection '
count' expects 6 bits on the pin connection, but pin connection's
VARREF 'seconds_disp' generates 7 bits.
                                : ... In instance
                                user_top_watch_v1
42 |         .count(seconds_disp)
   |         ^~~~~
   |         ... Use "/* verilator lint_off WIDTH */" and lint_on
   |         around source to disable this message.
```

Only the first of many such warnings is shown.

For each `editable_counter` instance, declare a correctly sized `logic` signal for its count output, for example:

```
1 logic [5:0] seconds; // This line is new
2 editable_counter #(
3     .N(60),
4     .WIDTH(6)
5 ) u_seconds (
```



```

6     .clk(clk),
7     .tick(seconds_tick),
8     .edit_mode(seconds_edit),
9     .inc(seconds_inc),
10    .dec(seconds_dec),
11    .count(seconds)    // This line has changed
12 );

```

At the end of your code, add:

```

1  // Zero-extend counter values to display outputs
2  assign hours_disp  = {2'b0, hours};
3  assign minutes_disp = {1'b0, minutes};
4  assign seconds_disp = {1'b0, seconds};

```

The other output ports of `user_top_watch_v1` have not yet been driven.

```

1  // Unused
2  assign led = 10'b0;
3  assign blank_hours = 1'b0;
4  assign blank_minutes = 1'b0;
5  assign blank_seconds = 1'b0;

```

Lint the design again; any warnings beyond the two below indicate a missed step.

```

root@a374924768bc:/workspace# verilator --Wall --lint-only -y rtl rtl/
user_top_watch_v1.sv
%Warning-UNUSED: rtl/user_top_watch_v1.sv:7:23: Signal is not used: '
    button'
                                     : ... In instance
                                     user_top_watch_v1
    7 |         input logic [3:0] button,
      |         ^~~~~~
      |         ... Use "/* verilator lint_off UNUSED */" and lint_on
      |         around source to disable this message.
%Warning-UNUSED: rtl/user_top_watch_v1.sv:8:23: Signal is not used: 'sw
    ,
                                     : ... In instance
                                     user_top_watch_v1
    8 |         input logic [9:0] sw,
      |         ^~
%Error: Exiting due to 2 warning(s)

```

Since the design does not use these inputs, we suppress the warnings explicitly.

```

1  /* verilator lint_off UNUSED */
2  input logic [3:0] button,
3  input logic [9:0] sw,
4  /* verilator lint_on UNUSED */

```

Re-run `verilator` and resolve any remaining warnings or errors.

We can now run the test bench, recalling that we use `-Y .sv -y rtl` and list only the test bench file.

```
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v1.v
./a.out
```

Examine the resulting .vcd file as usual. **The reporting requirements apply** (Section 5): include the waveforms in your report if you are satisfied they are correct. Then run `pytest` and verify that `test_user_top_watch_v1` passes.

Report 14

Reflect on the role of the test suite.

- **Manual testing cost.** Imagine verifying each version of the watch manually on the board as you build it up. For timekeeping alone you would need to observe seconds rolling over into minutes, minutes into hours, and hours back to zero. As more features are added, the number of button-press combinations to check grows rapidly. How long would thorough testing of a single version take, and would you want to repeat this for every version?
- **Regression testing.** Each new feature risks breaking code that already worked. Given your answer above, how valuable is it that the automated test suite can catch such a regression immediately?
- **Simulation speed.** The test bench overrides `CYCLES_PER_SECOND` to a small value. What would happen if the production value of 50 000 000 were used instead? What does this suggest about designing hardware with testability in mind from the outset?

7.14. Mode Selection (Watch V2)

Start by duplicating `rtl/user_top_watch_v1.sv`, renaming it `user_top_watch_v2.sv`, and changing the module name from `user_top_watch_v1` to `user_top_watch_v2`.

The standard development cycle (Section 5) applies; the test bench and automated tests for this version carry the suffix `_v2`.

At the end of `user_top_watch_v2`, add a comment marking the start of the mode selection logic, then add an instance of `edit_mode_selector` using the same approach as before.

```
1 // -----
2 // Mode Selection
3 // -----
4
5 logic [2:0] mode_enable;
6 edit_mode_selector #(
7     .HOLD_CYCLES(...) // Fill in, based on CYCLES_PER_SECOND
8 ) u_mode_selector (
9     // Fill in
10 );
```

Also instantiate a `pwm_generator` to produce the waveform that drives the display flash rate and duty cycle, connecting its output to a new `logic` signal `pwm_out`.

Finally, the `assign` statements that tie `blank_hours`, `blank_minutes`, and `blank_seconds` low should be replaced with appropriate expressions at the end of the module.

Note The `verilator` lint-off directive still suppresses warnings for `button` as a whole, even though we now use `button[3]`. Since the directive is just a comment, it is invisible to all other tools and has no effect on the design. Removing it would cause `verilator` to warn that the remaining buttons are unused, and adding dummy logic to silence that warning would be worse than simply acknowledging the unused inputs. We therefore leave the directive in place.

7.15. Settable Watch (V3)

Create `user_top_watch_v3` and add a comment marking the start of the edit logic. From here you are on your own: implement the full specification (Section 7.12), following the development cycle of Section 5 and the feature-layering approach of Section 7.13.

7.16. Accurate Time Setting (V4)

Just when you think you are finished, your friend complains that they cannot set the watch precisely: they have no control over when the seconds counter rolls over.

The desired feature is to realign the seconds tick when the user moves from editing seconds to editing minutes, so that the next tick occurs exactly one second later. This is achievable because the seconds counter is driven by a *restartable* rate generator.

Implement this feature in `user_top_watch_v4.sv`.

Report 15

Think carefully about which traces to include in your waveform report. All internal signals are visible, but there are many; include only those relevant to the feature being demonstrated.

Report 16

Does your design cause multiple resets? Why is this not a problem?

Hint Trigger the reset when editing seconds and the edit button is pressed.

7.17. Programming the Board

If your code passes all `pytest` tests, congratulations! Time to put it on the board.

Open the Quartus project from Section 6.8, copy your `rtl/*.sv` files into the project directory (overwriting the existing ones), then go to `Project >> Add/Remove Files` and add any new `*.sv` files; the simplest approach is to add all of them.

Open `top_de1_soc.sv` within Quartus. This is a local copy; do not edit the original in your VS Code workspace. Replace `user_top` with `user_top_watch_v4` and save.

Change the top-level module (`Assignments >> Settings >> General`) to `top_de1_soc`.

Compile; it should work. Because every module has been thoroughly linted, Quartus should produce no warnings about the design code, except for unused input and output pins.

Program it to the board and enjoy your digital watch.

Report 17

Reflect on your experience in previous lab classes. Were you typically able to get a design working on the first compile and board test? Which development workflow do you prefer: the VS Code simulation environment, or writing code directly in Quartus?

Report 18

Assume the 50 MHz clock on the DE1-SoC board is accurate to within 50 ppm. How many seconds might your watch drift over a 24-hour period?

A. Common Pitfalls

A.1. Installation

Git clone option not available Most likely, you need to install Git first.

Not offered to open the container Ensure the Dev Containers extension is installed and that you have opened the project folder itself in VS Code, not a parent folder. If you downloaded a zip file, opening a parent folder by mistake is a common cause of this.

A.2. Inexplicable Behaviour

If command-line tools fail to run, linting does not work, VS Code reports it cannot access files, or any other inexplicable behaviour occurs, **rebuild the dev container**: open the command palette (**Ctrl** + **Shift** + **P**), search for **Dev Containers: Rebuild Container**, and run it. If the problem persists, the cached image may be corrupted or incomplete — a network interruption during the initial build, for example, can leave packages only partially downloaded. In that case, use **Dev Containers: Rebuild Container Without Cache** instead, which discards the cache and rebuilds entirely from scratch.

Also ensure you are running the correct dev container: a blue label in the lower-left corner should read **Verilog (ELEN20006)**.

A.3. VS Code

New files do not appear in the Explorer Inside a dev container, VS Code's file-watching capability is limited, so newly created files do not always appear automatically in the Explorer pane. If a file you have just created is missing, click the refresh icon at the top of the Explorer panel.

Formatter silently does nothing The linter does not catch every error. If **Format Document** silently does nothing despite the file appearing error-free, run Verilator manually in the terminal:

```
1 verilator -Wall --lint-only -y rtl rtl/your_file.sv
```

Verilator will report the error, after which you can fix it and format normally.

A.4. SystemVerilog

'1 is not the same as 'd1 '1 is a fill literal that replicates 1 across every bit, giving all-ones. 'd1 is the decimal number 1 with a 32-bit default width. Consequently, `count + '1` adds all-ones, which is equivalent to subtracting one. Use `8'd1` or `WIDTH'(1)` when you mean the number one.

W'(N) silently truncates to zero In a counter with N states and W bits, the largest representable value is N-1, not N. Be very careful when writing

```
1 localparam logic [W-1:0] Max = W'(N);
```

When N is a power of two, W'(N) silently truncates to zero, since N requires one more bit than W provides.

logic x = y; is not a continuous assignment Writing `logic x = y;` assigns y to x once at time 0 and never again — it is *not* equivalent to `assign x = y;`, which updates x whenever y changes. Always use `assign` or `always_comb` for combinational logic.

A.5. Automated Tests

Module not found The automated tests have the required source file names hardcoded. Additional modules in separate files will not be loaded, causing your design to fail to compile. The implementation notes for each module specify exactly which modules to instantiate; these are always sufficient. Also check that each module's filename matches its module name exactly (for example, `editable_counter` must be in `rtl/editable_counter.sv`).

Tests are skipped If the test output shows tests as skipped rather than passed or failed, the source file for the module under test is missing from the `rtl` directory. Check that the filename matches the module name exactly and has the `.sv` extension.

A.6. Simulation

Forgetting -g2012 Omitting `-g2012` causes `iverilog` to parse files as Verilog rather than SystemVerilog, producing confusing syntax errors on perfectly valid SystemVerilog constructs. Always include `-g2012` when compiling `.sv` files.

Excessive runtime warnings Passing a wildcard such as `rtl/*.sv` to `iverilog` compiles every file in the directory, including modules from previous tasks. These modules contain `unique case` statements that may fire at runtime, producing spurious warnings that are difficult to distinguish from real ones. To avoid this, use the library flags instead and pass only the test bench file explicitly:

```
iverilog -Wall -g2012 -Y .sv -y rtl tb/wave_user_top_watch_v1.v
```

The `-Y .sv` flag must come before `-y rtl`.

A.7. VaporView

Waveforms do not update after recompiling To have VaporView reload the waveform automatically each time you run `./a.out`, tick the `Auto Reload` checkbox in the VaporView menu bar. If you forgot to do this before running your simulation, tick it now and run `./a.out` again. Note that this only works if the simulation writes to the same `.vcd` filename each run.

VaporView prompts to save settings When a VaporView window is open, and you attempt a file operation elsewhere in VS Code, VaporView may prompt you to save its settings. Close the VaporView window; it is fine not to save.

Adding a signal from the Netlist View adds the wrong trace If the wrong trace appears after clicking a signal in the Netlist View, the most likely cause is a name clash: two signals in your design share the same identifier. Prefix all module instantiations with `u_` (e.g. `u_restartable_rate_generator`) and check for other duplicate names.

A subtler version of this bug arises when two different modules each instantiate a submodule under the same instance name, for example both `module_a` and `module_b` contain an instance called `u_counter`. Even though the instances are in different scopes, VaporView may confuse them and display the wrong trace. While tedious, the safest approach is to avoid reusing any instance name across the entire project. Alternatively, find a waveform viewer you are comfortable with.

A.8. Windows Filesystem Performance

On Windows, your project folder normally lives on the Windows filesystem, while the dev container runs inside WSL2; every file operation therefore crosses between the two, which is markedly slower than a native Linux filesystem. Expect linting, formatting, and test runs to feel noticeably sluggish. The same boundary makes file corruption more likely, particularly if VS Code, Docker, or the host machine is interrupted mid-write; Appendix A.2 covers the fix: rebuilding the dev container. Confident students can, and should, work under WSL directly instead, storing the project inside the WSL filesystem rather than on the Windows drive. We do not give step-by-step instructions here: this route is for students already comfortable enough with WSL to set it up unaided.

B. SystemVerilog Initialisation

Immediate initialisation — giving a variable a value at the point of declaration — behaves differently from an `initial` block:

```
1 logic [3:0] count = '0;
```

This gives the variable a defined value before simulation begins. An `initial` block, by contrast, executes at time 0 in simulation, with no guaranteed order relative to other `initial` blocks. In particular, if a test bench uses an `initial` block to check that a variable has been correctly initialised by another `initial` block, there is a race condition. For this

project, use immediate initialisation in port declarations or module bodies, and reserve `initial` for test benches.

A related change concerns `always @(*)`, which is no longer guaranteed to fire at time zero. To be precise, in Verilog, writing `reg y = 0;` will cause `y` to change from unknown (`1'bx`) to zero (`1'b0`) at time 0, and this change will trigger relevant `always @(*)` blocks. However, in SystemVerilog, immediate initialisation takes place before simulation starts, so there is no transition at time zero to trigger the `always @(*)` block. This is demonstrated in `demos/simulation.v`; see the comments at the top of the file for how to compile it. The fix is to use `always_comb` instead, which is guaranteed to run at time zero irrespective of whether any transitions were detected.

C. Inside the Development Environment

This appendix has two interwoven purposes: it explains the toolchain (what each tool is, how they fit together, and how they work), and it documents installation. For historical reasons the installation instructions assume Windows, where installation is considerably more involved and therefore worth documenting in detail. On Linux, most tools require little more than a single `apt-get install` command. With VS Code dev containers now providing a seamless Linux environment on any host, there is no good reason to persist with a native Windows setup.

You do not need to read this appendix, but if you are curious about how the environment works, read on. Understanding what lies beneath makes you a more capable engineer: these tools are built on open-source software, which invariably comes with bugs and incomplete documentation, and some knowledge of the internals will help you diagnose problems when they arise.

C.1. Toolchain

C.1.1. Installing Icarus Verilog

Icarus Verilog (`iverilog`) is the open-source Verilog simulator used by the automated test suite and the waveform test benches.

On Windows, installation is done through MSYS2, as described below.

MSYS2 Many serious software development tools originated on Unix and have never been officially ported to Windows. One common solution is MSYS2, which provides a Unix-like shell and the MinGW compiler toolchain. MinGW (Minimalist GNU for Windows) is a port of the GNU Compiler Collection (GCC) and associated build tools to Windows; crucially, it produces native Windows executables rather than emulating Unix at runtime, so the resulting programs run directly on Windows without any Unix dependency. Icarus Verilog is one such tool: developed primarily on Unix, it is made available on Windows by compiling it with MSYS2's MinGW toolchain.

As well as the MinGW toolchain, MSYS2 includes the `pacman` package manager. A package manager eliminates the need to compile software from source: pre-built binaries are downloaded and installed with a single command, with dependencies resolved automatically. In practice, however, this is not always reliable. At the time of writing, the `iverilog` package has a bug in which not all dependencies are declared correctly, meaning some

must be installed manually. To install MSYS2, download the installer from msys2.org, run it, and accept the default installation path (`C:\msys64`).

MSYS2 provides several environments, each targeting a different architecture and C runtime library. The main ones are: **MSYS**, the Unix-like shell environment used for running MSYS2's own tools; **MINGW32** and **MINGW64**, which produce 32-bit and 64-bit Windows executables respectively, linking against the legacy MSVCRT C runtime; **UCRT64**, which produces 64-bit executables linking against the Universal C Runtime (UCRT); and **CLANG64**, which also targets UCRT but uses the Clang/LLVM compiler instead of GCC. UCRT was introduced with Windows 10 and is now the standard C runtime for modern Windows development, making UCRT64 the recommended environment for GCC-based work. All subsequent steps should be carried out in the UCRT64 terminal.

Once installed, open the MSYS2 UCRT64 terminal and update the package database:

```
1 pacman -Syu
```

The flags have the following meanings: **-S** invokes the sync operation, which is used for installing and upgrading packages; **-y** refreshes the local package database by downloading the latest package lists from the remote servers; and **-u** upgrades all currently installed packages for which a newer version is available. Together, **-Syu** ensures the package database is current before performing the upgrade.

If the terminal closes mid-update, reopen it and run the command again until no further updates are available.

MSYS2 Directory Layout MSYS2 installs to `C:\msys64`, with a subdirectory for each environment. Within the MSYS2 UCRT64 terminal, the Unix root `/` maps to `C:\msys64`, so a path such as `/ucrt64/bin` refers to `C:\msys64\ucrt64\bin` on the Windows filesystem. The most important directories are:

/ucrt64/bin Executables for the UCRT64 environment, including `iverilog` and `vvp` once installed.

/ucrt64/lib Libraries for the UCRT64 environment, linked against at compile time.

/ucrt64/include Header files for the UCRT64 environment.

/usr/bin Unix shell utilities (e.g. `bash`, `ls`, `grep`) used by the MSYS2 shell itself, regardless of which environment is active.

/var/lib/pacman The `pacman` package database, recording which packages are installed and their versions.

The other environment directories (`/mingw32`, `/mingw64`, `/clang64`) follow the same internal structure as `/ucrt64` but are not needed here.

Icarus Verilog With MSYS2 updated, open the UCRT64 terminal. Search for available `iverilog` packages with:

```
1 pacman -Ss iverilog
```

The **-Ss** flag searches the package database. The output will list several packages; the one to install is from the `ucrt64` repository. Specifying the repository explicitly in the install command ensures the correct package is selected:

```
1 pacman -S ucrt64/mingw-w64-ucrt-x86_64-iverilog
```

The package name follows a consistent naming convention used throughout the MSYS2 repositories: `mingw` stands for Minimalist GNU for Windows; `w64` indicates 64-bit Windows; `ucrt` identifies the Universal C Runtime; and `x86_64` is the CPU architecture. The final component, `iverilog`, is the package name itself.

To verify the installation, run:

```
1 iverilog -V
```

The version number should be printed. The `iverilog` executable is located in `C:\msys64\ucrt64\bin`. This directory is added to the `PATH` automatically within the UCRT64 terminal, which is why the command above works without specifying a full path. However, when `iverilog` is invoked from outside the MSYS2 terminal — for example, by a Python-based test runner or build script — the MSYS2 `PATH` settings are not inherited, and the full path to the executable must be provided explicitly. Knowing where `iverilog` is installed is therefore important for configuring such tools.

Although `iverilog` now runs, simulations that use system tasks such as `$dumpfile` and `$dumpvars` will fail. The reason is that `vvp` (the Icarus Verilog simulation runtime, described in the next section) needs to load `ucrt64/lib/ivl/system.vpi`, a plugin that provides these system tasks, but `system.vpi` in turn depends on `zlib` and `bzip2` at runtime. These dependencies are not declared in the `iverilog` package, so the plugin fails to load and `vvp` reports:

```
1 system.vpi failed to open using dlopen():
2 The specified module could not be found.
```

The fix is to install the two missing packages manually:

```
1 pacman -S ucrt64/mingw-w64-ucrt-x86_64-zlib
2 pacman -S ucrt64/mingw-w64-ucrt-x86_64-bzip2
```

More generally, when a DLL fails to load, `ntldd` can be used to list all DLL dependencies of a binary and identify which ones are missing. Install it first:

```
1 pacman -S ucrt64/mingw-w64-ucrt-x86_64-ntldd
```

Then run it against the library in question:

```
1 ntldd /ucrt64/lib/ivl/system.vpi
```

Each dependency is listed with its resolved path, or flagged as `not found` if missing. This is how the absent `zlib` and `bzip2` libraries were identified.

C.1.2. Using Icarus Verilog

Icarus Verilog compiles Verilog source files into an intermediate representation, which is then executed by the `vvp` runtime. The basic workflow is:

```
1 iverilog -Wall -o sim tb_my_module.v my_module.v
2 vvp sim
```

Those with a Unix background will not be surprised that `-o` can be omitted, in which case the output file is named `a.out` by default — the same convention used by C compilers. Since `a.out` is a script with a shebang line, it can be executed directly:

```
1 ./a.out
```

Running `head a.out` reveals the shebang line and confirms that `vvp` is invoked when the output file is executed.

The most commonly used flags are summarised below.

-Wall Enables all warnings.

-o file Specifies the output file name; defaults to `a.out`.

-g2012 Enables IEEE 1800-2012 (SystemVerilog) language features. Required when compiling `.sv` source files.

-Y ext Adds *ext* as a recognised file extension when searching library directories. Use `-Y .sv` alongside `-y` to allow `iverilog` to find `.sv` files automatically.

-y dir Adds *dir* as a library directory. When a module instantiation cannot be resolved from the files listed on the command line, `iverilog` searches library directories for a file whose name matches the module name.

-s module Specifies the top-level module. Only modules reachable from this root are compiled, which avoids pulling in unrelated modules from library directories.

C.1.3. Verilator

Verilator is an open-source SystemVerilog linter and simulator. In this course it is used purely as a linter, and can be installed via MSYS2 in the same way as `iverilog`.

The VS Code extension catches some issues, but Verilator catches considerably more, particularly mismatched bit widths that cause implicit truncation or sign extension. The recommended workflow is therefore to edit in VS Code and then run Verilator from the command line. Precision in specifying bit widths may initially feel laborious, but it quickly becomes second nature and can save hours of debugging; aim for zero Verilator warnings and errors.

For the purposes of this course, Verilator is invoked in lint-only mode, which checks for errors without invoking the simulator. The most commonly used flags are:

--lint-only Checks for lint errors without generating any simulation output.

--Wall Enables all warnings, including bit-width mismatches and implicit truncations.

-y dir Adds *dir* as a library directory, searched when resolving module instantiations.

A typical invocation for linting a top-level file with additional modules in `rtl/` is:

```
1 verilator --lint-only --Wall -y rtl rtl/top_module.sv
```

C.1.4. Universal Ctags

Ctags is a tool that indexes source code by scanning files and recording the location of every symbol (module names, signal declarations, port lists, and so on) into a tags file. Editors use this index to provide navigation features: jumping to a symbol’s definition and showing its declaration on hover. One VS Code Verilog extension relies on `ctags` for these capabilities; without it, hovering over a signal name will not show where it was declared, and go-to-definition will not work.

The original `ctags` program has long been superseded; the current actively maintained implementation is Universal Ctags. Since MSYS2 is already installed as part of the Icarus Verilog setup (see Appendix C.1.1), it can be installed via `pacman` in the UCRT64 terminal:

```
1 pacman -S ucrt64/mingw-w64-ucrt-x86_64-universal-ctags
```

Once installed, the path to the `ctags` executable must be provided to the relevant VS Code Verilog extension; this is covered in Appendix C.2. From the directory layout described earlier, it will be at `C:\msys64\ucrt64\bin\ctags.exe`.

C.1.5. Python and Virtual Environments

The `cocotb` testing framework requires Python; Python must therefore be installed. Visit python.org and download the standalone installer; when prompted during installation, enable the option to add Python to the system `PATH`. Python also offers a “Python Install Manager”, which manages multiple Python versions side by side: if you already use Python for other projects and need several versions available at once, this is the better choice. For a fresh installation, the standalone installer is simpler.

In general, use the latest stable release, but it is worth understanding Python’s version numbering first. Version numbers take the form `major.minor.micro`: in `3.14.2`, for example, the minor version is 14 and the micro version is 2. A new micro release is not a concern. If a new minor release has just come out (for example, 3.15 following 3.14), check the `cocotb` GitHub page to confirm support before installing it; third-party packages sometimes take a few weeks or months to catch up, and an incompatibility of this kind can be difficult to diagnose. This is part of the motivation behind tools such as `conda` (distributed as Anaconda, the full scientific Python distribution, or Miniconda, the minimal installer): `conda` manages both the interpreter and its packages together, making it straightforward to work on projects that require different Python versions. It is widely used in data science for precisely this reason. For most purposes, a single stable installation is all that is needed.

Virtual Environments Python packages can be installed into the system-wide Python installation, or into a *virtual environment*: a self-contained directory holding its own copy of the Python interpreter and a private set of installed packages. The system-wide installation is best avoided for project work: if you later install or upgrade a package for another project, you may inadvertently affect what is available here, and the resulting incompatibility can be difficult to trace. A virtual environment keeps each project’s dependencies isolated and the setup reproducible.

For projects where a Python application or library is the product being developed and distributed, a dependency manager such as Poetry is the standard choice at the time of

writing: it handles virtual environments automatically, records exact dependency versions, and manages packaging. When the goal is to use a set of Python tools rather than to build and distribute an application, a manually managed virtual environment is the simpler approach.

The terminal commands in this section must be run in Windows Command Prompt or PowerShell, not in an MSYS2 terminal. The MSYS2 terminal has its own Python environment and file path conventions that will interfere with the Windows Python installation.

A virtual environment is conventionally placed in the *project root*: a dedicated directory that serves as the home for all project files, typically organised into subdirectories such as `src/` for source files and `tests/` for tests. When starting a project, the first step is to create this root directory. To create a virtual environment within it, navigate to the root directory in a terminal and run:

```
1 python -m venv .venv
```

This creates a `.venv` subdirectory containing the isolated Python installation. To activate the environment:

```
1 .venv\Scripts\activate
```

The activate script works by prepending the virtual environment's `Scripts` directory to the `PATH` environment variable. When you subsequently type `python`, the shell searches `PATH` in order and finds the virtual environment's interpreter first, rather than the system-wide one. Once activated, the shell prompt changes to show `(.venv)`, confirming that `python` and `pip` now refer to the local environment.

Confirming the Active Environment The shell prompt showing `(.venv)` is a helpful indicator, but it is worth knowing how to confirm directly which Python executable is in use. In `cmd`:

```
1 where python
```

In PowerShell, `where` is an alias for `Where-Object` and does not search the `PATH`; use `where.exe` or `Get-Command` instead:

```
1 where.exe python
2 Get-Command python
```

If the virtual environment is active, the path returned will point to `.venv\Scripts\python.exe` inside the project root. If it points to the system-wide installation instead, the environment has not been activated.

C.1.6. pytest

`pytest` is a widely used Python testing framework. It discovers tests automatically by scanning for files and functions whose names follow the `test_` naming convention, runs them, and reports the results in a format that makes it easy to see which tests passed and which failed.

With the virtual environment activated, install `pytest`:

```
1 pip install pytest
```

To run all discovered tests, invoke **pytest** from the project root:

```
1 pytest
```

The **-v** flag enables verbose output, printing the name of each test as it runs rather than just a dot:

```
1 pytest -v
```

To run only the tests in a specific file:

```
1 pytest tests/test_my_module.py
```

To run only tests whose names contain a given string, use the **-k** flag:

```
1 pytest -k counter
```

This runs any test whose name contains **counter**.

When there are many failures, the **-x** flag stops the run after the first one, making it easier to focus on a single problem at a time:

```
1 pytest -x
```

Once a failure has been fixed, **--lf** (last failed) re-runs only the tests that failed in the previous run, avoiding the need to run the entire suite:

```
1 pytest --lf
```

By default, **pytest** captures output from **print** statements and only shows it when a test fails. The **-s** flag disables this, printing output immediately as the test runs, which is useful when debugging:

```
1 pytest -s
```

Flags can be combined freely; for example, **pytest --lf -v -s** re-runs only the previously failing tests with verbose output and captured printing disabled.

C.1.7. cocotb

Installation **cocotb** (Coroutine-based Co-simulation Test Bench) is a Python library for writing hardware test benches for Verilog or VHDL designs. Python is considerably more expressive than Verilog for this purpose, providing access to the full Python ecosystem: data structures, file I/O, numerical libraries, and much more. What makes **cocotb** particularly elegant is how this integration works: using the VPI (Verilog Procedural Interface), it embeds a Python interpreter directly into the running simulator. Test bench code is written as Python coroutines that execute concurrently with the simulation, suspending and resuming in step with simulation events. A test can wait for the next rising clock edge, read a signal value, drive an input, wait again, and assert the result, all expressed as ordinary Python statements.

Used on its own, `cocotb` requires a good deal of manual work: test discovery, invocation, and reporting must all be managed explicitly, and features such as running only a subset of tests require custom code. Pytest addresses this. The integration with pytest means that each `cocotb` test function is recognised as a pytest test, and the full suite can be run, filtered, and reported through the familiar `pytest` command and its flags. The division of responsibility is clean: pytest handles the user interface, while `cocotb` handles the hardware simulation.

Install `cocotb` into the virtual environment:

```
1 pip install cocotb
```

To verify the installation:

```
1 python -c "import cocotb; print(cocotb.__version__)"
```

If successful, this prints the `cocotb` version number.

Usage Once the test suite has been set up, running it is straightforward: open a Command Prompt or PowerShell, navigate to the project root, activate the virtual environment, and invoke `pytest`. Behind the scenes, the test setup code must specify which simulator to use and where to find it; when using Icarus Verilog installed via MSYS2, this is `C:\msys64\ucrt64\bin`.

The MSYS2 UCRT64 terminal prepends `C:\msys64\ucrt64\bin` to `PATH`, which is why `iverilog` can be found when running commands there. Outside the MSYS2 terminal, this is not in place. When `cocotb` invokes `iverilog` or `vvp` as a subprocess during a test run, Windows must be able to locate those executables and resolve their DLL dependencies. Since `C:\msys64\ucrt64\bin` is not on the system `PATH` by default, the test setup code (typically `conftest.py`) must prepend it at startup:

```
1 import os
2 # Add msys64 bin directory to PATH so iverilog can be found
3 os.environ["PATH"] = (
4     r"C:\msys64\ucrt64\bin" + os.pathsep + os.environ["PATH"]
5 )
```

The `r` prefix marks the string as a raw string, preventing Python from interpreting the backslashes as escape sequences. `os.pathsep` is the platform-appropriate path separator (a semicolon on Windows); prepending rather than appending ensures the MSYS2 executables take precedence if another `iverilog` is already on the system `PATH`.

C.1.8. Why Not VUnit?

VUnit is another hardware verification framework, and was considered before `cocotb` was adopted. It uses Python to configure the test run, but the test benches themselves are written in HDL. This creates two problems. First, VUnit uses its own test runner rather than `pytest`, so none of `pytest`'s infrastructure (test filtering, `--lf`, concise summaries) is available without additional work. Second, and more fundamentally, HDL test benches are far less expressive than Python: Verilog lacks dynamic data structures, rich string handling, and file I/O, while SystemVerilog recovers some of this at the cost of considerable complexity. Writing test benches in Python directly, as `cocotb` allows, is simpler, more expressive, and easier to debug.

C.2. VS Code

Visual Studio Code (VS Code) is a professional, open-source code editor developed by Microsoft. It is widely used across industry and academia for software development in virtually every language. Although Quartus includes a text editor, it is primarily a synthesis and place-and-route tool; its editing experience is minimal by comparison. VS Code is purpose-built for editing code, and offers several advantages for writing SystemVerilog:

- **Syntax highlighting and error underlining.** Verilog extensions provide syntax colouring and underline errors as you type.
- **Go-to-definition and hover documentation.** Pressing **F12** or **Ctrl**+clicking a signal or module name navigates to its declaration; hovering shows the declaration inline.
- **Integrated terminal.** A terminal panel within VS Code allows you to run `iverilog`, `pytest`, and other tools without leaving the editor.
- **Split editor.** Multiple files can be viewed side by side, making it easy to reference one module's interface while writing another.
- **File diff and selective merge.** VS Code can diff any two files and copy individual changes from one to the other with a single click. This is particularly useful in feature-layered designs: if a bug is found in Watch V1, the fix can be selectively ported to V2, V3, and V4 without overwriting unrelated changes.
- **Multi-cursor editing.** Multiple cursors can be placed simultaneously, allowing repeated structures with minor name variations (common in Verilog) to be edited in one step.
- **Project-wide search.** **Ctrl** + **Shift** + **F** searches across all files in the project, making it easy to find every use of a signal name or module.
- **Git integration.** Source control is built in, making it straightforward to stage, commit, and review changes.
- **Dev container support.** VS Code can open a project inside a Docker container, providing a consistent, pre-configured Linux environment on any host operating system.

Quartus remains necessary for synthesis and programming the board, but all code editing and simulation should be done in VS Code. In larger professional projects, `iverilog` is typically replaced by a commercial simulator such as Questa, VCS, or Xcelium, but the workflow is the same: the simulator is invoked from a script rather than a GUI, and a test bench framework checks the results. The workflow used in this course is therefore a realistic representation of professional practice.

VS Code can be further enhanced through extensions. The following sections describe the extensions used in this course; the dev container installs them automatically, except where noted.

C.2.1. Verible

Verible (`chipsalliance.verible`) is Google's open-source SystemVerilog formatter and linter. It underlines lint violations as you type and, with the settings below, formats files on save. The lint rules it enforces are loaded from `.rules.verible_lint` in the project root, as specified by the `verible.arguments` setting below.

To configure manually, add the following to `.vscode/settings.json`:

```
"[verilog]": {
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "chipsalliance.verible",
  "editor.colorDecorators": false
},
"[systemverilog]": {
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "chipsalliance.verible",
  "editor.colorDecorators": false
},
"verible.format.enabled": true,
"verible.lint.enabled": true,
"verible.arguments": [
  "--rules_config=.rules.verible_lint",
  "--push_diagnostic_notifications"
],
```

VS Code renders a small inline colour swatch next to any value it interprets as a CSS colour code. Verilog's timing delays (`#10`) and hex literals (`8'hFF`) can trigger this, scattering distracting swatches through the file. Setting `editor.colorDecorators` to `false` disables this behaviour for Verilog and SystemVerilog files.

Mac Workaround The Verible extension is designed to download `verible-verilog-ls`, the binary that provides its language services, but on Mac this does not appear to happen automatically. As a workaround, the Dockerfile detects the host architecture at build time and installs the appropriate `verible-verilog-ls` binary into `/usr/local/bin`, where the extension can find it; only `amd64` and `arm64` are supported. A side benefit is that the full suite of Verible command-line tools, including `verible-verilog-format` and `verible-verilog-lint`, becomes available inside the container.

C.2.2. SystemVerilog

The `eirikpre.systemverilog` extension provides go-to-definition, hover documentation, and symbol indexing. Pressing `F12` or `Ctrl`+clicking a signal or module name navigates to its declaration; hovering shows the declaration inline.

The `systemverilog.includeIndexing` setting controls which files the extension indexes; without it, cross-file references may not resolve correctly. To configure manually:

```
"systemverilog.includeIndexing": [
  "**/*.{sv,v,svh,vh}"
],
```

The documentation suggests all matching files are indexed automatically; in practice, however, the extension appears to index a file only once it has been opened. This means that pressing **F12** on a module instantiation may not navigate to the module's definition if the file containing that definition has not yet been opened.

C.2.3. Python and Ruff

The `ms-python.python` extension provides Python language support and `pytest` integration, making tests discoverable and runnable from the Testing view.

The `charliermarsh.ruff` extension (Ruff) is a fast Python formatter and linter; it formats Python files on save via the settings below.

To configure manually, add the following to `.vscode/settings.json`:

```
"[python]": {
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "charliermarsh.ruff"
},
"python.testing.unittestEnabled": false,
"python.testing.pytestEnabled": true,
```

Setting `pytestEnabled` to `true` (and `unittestEnabled` to `false`) tells VS Code's built-in Testing view to use `pytest` as the test runner; tests are then discoverable and runnable directly from the Testing panel.

The Python virtual environment path must also be recorded. Without it, the linter will report missing modules when you open the project's Python files, and the VS Code test runner may invoke the global Python, which does not have `pytest` installed. The easiest way is via **Ctrl+Shift+P** then typing and selecting **Python: Select Interpreter**, which writes it to `.vscode/settings.json` automatically. Alternatively, specify it manually as shown below, adjusting the path as needed.

```
"python.defaultInterpreterPath": "/opt/venv/bin/python",
```

C.2.4. VaporView

VaporView (`lramseyer.vaporview`) is a waveform viewer that runs directly inside VS Code. It opens `.vcd` files produced by `iverilog` without needing a separate application. See Appendix A.7 for common VaporView issues.

C.2.5. Verilog HDL

The `mshr-h.veriloghdl` extension provides go-to-definition and symbol indexing for Verilog and SystemVerilog by invoking Universal Ctags in the background. Unlike `eirikpre.systemverilog`, which maintains its own index, this extension delegates symbol lookup to Ctags. This extension has not declared that it is able to run inside a dev container, so it is only available on a native Windows install. The `verilog.ctags.path` setting tells the extension where to find the Ctags binary; add the following to `.vscode/settings.json`:

```
"verilog.ctags.path": "C:/msys64/ucrt64/bin/ctags",
```

C.2.6. MSYS2 Terminal Profile

On Windows, the integrated terminal defaults to PowerShell or Command Prompt, neither of which has access to MSYS2 tools such as `iverilog`. Adding a terminal profile for MSYS2 allows you to open an MSYS2 UCRT shell directly from within VS Code, without leaving the editor. Once configured, it appears as an option in the terminal dropdown (the + button in the Terminal panel). To configure manually, add the following to `.vscode/settings.json`:

```
"terminal.integrated.profiles.windows": {
  "MSYS2 UCRT": {
    "path": "cmd.exe",
    "args": [
      "/c",
      "C:\\msys64\\msys2_shell.cmd -defterm -here -no-start -
        ucrt64"
    ]
  }
},
```

The flags passed to `msys2_shell.cmd` control how the shell launches:

- defterm** Use the calling process's terminal rather than opening a new mintty window. Without this, MSYS2 would launch a separate window instead of appearing inside VS Code.
- here** Start the shell in the current working directory rather than the MSYS2 home directory.
- no-start** Run the shell process directly rather than via the Windows `start` command. This keeps it attached to VS Code's terminal panel as expected.
- ucrt64** Use the UCRT64 subsystem. MSYS2 provides several subsystems; UCRT64 is the modern, recommended choice for new installations.

This is not needed when using the dev container, since the integrated terminal is already a Linux shell with all tools available.

C.3. Dev Container

Docker is software for running *containers*: isolated Linux environments with their own filesystem and installed software, but which share the host operating system's kernel rather than emulating hardware as a full virtual machine would. This makes containers far lighter than VMs: they start in seconds and consume only the resources the running software actually needs.

Docker works by building an *image* from a **Dockerfile**, a script that specifies a base operating system and a sequence of setup steps such as installing packages and copying files (an example is given below). Building the image the first time amounts to a fresh installation of whichever operating system the **Dockerfile** requests and takes several minutes. Once built, however, the image is cached, and running a container from it is near-instant since all the software is already in place.

A *dev container* is a container configured as a development environment, defined by an open specification supported by several tools and platforms, including VS Code, GitHub

Codespaces, and JetBrains IDEs. The configuration consists of a `Dockerfile` and a `devcontainer.json` file. The VS Code Dev Containers extension reads these files, builds the image if it has not been built before, and launches a container from it. The integrated terminal, extensions, and linting all run inside this Linux container.

The project files remain on the host machine and are mounted into the container at startup, making them simultaneously accessible inside and outside: a waveform viewer installed on the host, for example, can open `.vcd` files the moment they are produced inside the container.

There are several benefits to this approach:

- **Reproducibility.** Every user gets the same operating system, tool versions, and extension configuration, regardless of their host machine.
- **Native Linux environment.** The tools used in this course are designed for Linux; installation is straightforward, whereas on Windows each requires workarounds such as MSYS2.
- **Uniformity.** On a native Windows setup, development requires switching between an MSYS2 shell and a standard shell; the two use different commands for the same operations and different path separators.
- **No Python virtual environment needed.** The container is itself an isolated environment, so Python packages can be installed globally within it without affecting the host; a separate `venv` is unnecessary.

Docker Desktop must be installed on the host; VS Code then handles the rest automatically.

C.3.1. Configuration

The dev container is configured by two files in the `.devcontainer` directory.

Dockerfile The `Dockerfile` defines the Linux image:

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN sed -i 's|archive.ubuntu.com|mirror.aarnet.edu.au|' /etc/apt/
    sources.list.d/ubuntu.sources

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential \
    git \
    wget \
    curl \
    make \
    python3 \
    python3-pip \
    python3-venv \
    libpython3-dev \
    iverilog \
    verilator \
    perl-doc \
```

```

yosys \
graphviz \
z3 \
locales \
ca-certificates \
&& rm -rf /var/lib/apt/lists/*

RUN locale-gen en_AU.UTF-8
ENV LANG=en_AU.UTF-8
ENV LC_ALL=en_AU.UTF-8

WORKDIR /workspace

RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:${PATH}"
RUN pip install --no-cache-dir --upgrade pip \
    && pip install --no-cache-dir cocotb==2.0.1 pytest==9.0.3 pytest-
    html==4.2.0 click==8.3.2

COPY --chmod=0755 sby/bin/sby /usr/local/bin/sby
COPY sby/share /usr/local/share

ARG TARGETARCH
RUN --mount=type=bind,source=arch,target=/archs \
    if [ -d "/archs/$TARGETARCH" ]; then \
        install -m 0755 /archs/$TARGETARCH/bin/* /usr/local/bin/; \
    else \
        echo "No binaries for TARGETARCH=$TARGETARCH"; \
    fi

```

A line-by-line explanation follows, with additional notes afterwards.

FROM ubuntu:24.04 Base the image on Ubuntu 24.04 LTS.

ENV DEBIAN_FRONTEND=noninteractive Prevents **apt-get** from prompting for user input during package installation.

RUN sed Edits a config file so that Ubuntu packages are fetched from a nearby mirror rather than an overseas server, which can significantly speed up the container build.

RUN apt-get install Installs the required tools. The **--no-install-recommends** flag suppresses optional packages, and the trailing **rm -rf /var/lib/apt/lists/*** removes the package index to keep the image size small.

RUN locale-gen / ENV LANG / ENV LC_ALL Sets the locale to Australian UTF-8.

WORKDIR /workspace Sets the default working directory inside the container.

RUN python3 -m venv / ENV PATH / RUN pip install Ubuntu does not allow packages to be added to the global Python environment, so a virtual environment is created at **/opt/venv**, keeping it out of the project directory. Prepending **/opt/venv/bin** to **PATH** ensures the correct Python is used even if the virtual environment is not explicitly activated. The required packages are then installed with pinned version numbers (e.g. **cocotb==2.0.1**, **pytest==9.0.3**) to guarantee a reproducible environment; without pinning, a future rebuild could pull in a newer version with different behaviour.

COPY sby The formal verification tool **sby** requires **yosys** and **z3**, both installed via the package manager above. **sby** itself is a collection of Python scripts; installing from source merely copies them into `/usr/local/bin` and `/usr/local/share`. A snapshot of these files is therefore included in the repository, and the two **COPY** lines place them directly into position without a source build.

ARG TARGETARCH / RUN install Docker exposes the build architecture as a build argument. The repository bundles pre-compiled Verible binaries for each supported architecture under an `arch/` directory; the **RUN** command mounts that directory and installs the appropriate binaries into `/usr/local/bin`, or prints a message and skips the step if the architecture is not recognised.

The container runs as root. This is acceptable because the container is ephemeral and can be rebuilt if anything goes wrong. The more conventional approach of creating a dedicated user introduces a file-permission problem on Linux hosts, where file ownership is determined numerically by user ID: unless the container user's ID happens to match the host user's ID, the user cannot access their own mounted files.

Verible is not part of the Ubuntu package manager and therefore cannot be installed via **apt-get**. Because a Docker container shares the host's kernel rather than virtualising it, any binary must match the host's CPU architecture; the **ARG TARGETARCH / RUN install** step above selects the correct binary at build time. The full suite of Verible command-line tools is therefore available inside the container. Verible is also installed as a VS Code extension, which provides editor integration independently.

Each **RUN** command creates a new image layer. If one **RUN** creates files that a later **RUN** deletes, both layers are retained, so the deleted files still contribute to the image size. Related commands are therefore commonly daisy-chained within a single **RUN** using **&&**, as in the **apt-get** line above. Our configuration does not currently encounter this problem elsewhere, but it is worth being aware of.

devcontainer.json The `devcontainer.json` file tells VS Code how to build and configure the container:

```
{
  "name": "Verilog (ELEN20006)",
  "build": {
    "dockerfile": "Dockerfile"
  },
  "customizations": {
    "vscode": {
      "extensions": [
        "chipsalliance.verible",
        "eirikpre.systemverilog",
        "ms-python.python",
        "lramseyer.vaporview",
        "charliermarsh.ruff"
      ],
      "settings": {
        "python.defaultInterpreterPath": "/opt/venv/bin/python",
        "[python]": {
          "editor.formatOnSave": true,
          "editor.defaultFormatter": "charliermarsh.ruff"
        },
        "[verilog]": {
          "editor.formatOnSave": true,
          "editor.defaultFormatter": "chipsalliance.verible",
          "editor.colorDecorators": false
        },
        "[systemverilog]": {
          "editor.formatOnSave": true,
```

```

        "editor.defaultFormatter": "chipsalliance.verible",
        "editor.colorDecorators": false
    },
    "verible.format.enabled": true,
    "verible.lint.enabled": true,
    "verible.arguments": [
        "--rules_config=.rules.verible_lint",
        "--push_diagnostic_notifications"
    ],
    "systemverilog.includeIndexing": [
        "**/*.sv,v,svh,vh"
    ],
    "python.testing.unittestEnabled": false,
    "python.testing.pytestEnabled": true,
    "workbench.editorAssociations": {
        "*.md": "vscode.markdown.preview.editor"
    }
}
}
},
"workspaceFolder": "/workspace",
"mounts": [
    "source=${localWorkspaceFolder},target=/workspace,type=bind"
],
"remoteUser": "root"
}

```

build.dockerfile Points VS Code at the Dockerfile to build the image from.

customizations.vscode.extensions Lists the extensions to install automatically inside the container. The **settings** block configures these extensions as described in the preceding sections.

workspaceFolder The path inside the container at which the project is available.

mounts Bind-mounts the local project directory into **/workspace** inside the container, so edits made in VS Code are immediately reflected on the host machine.

remoteUser Runs the container as **root** (superuser). This may sound dangerous, but the only files at risk are the project files mounted from the host (files you already have full access to). Everything else inside the container is ephemeral: if the container misbehaves for any reason, simply rebuild it (see Appendix A.3).

Although **devcontainer.json** supports a **postCreateCommand** field for running commands after the container starts, it has been deliberately left unused. When present, VS Code opens a terminal showing those commands as they run, which is distracting for students. Performing all setup inside the Dockerfile keeps the experience clean: by the time the container opens, everything is already in place.

D. Version History

Version	Date	Changes
1.5	2026-09-05	Windows performance, and documentation improvements.
1.4	2026-05-07	Documentation improvements.
1.3	2026-05-02	Added linting and compilation checks to the test suite.
1.2	2026-05-01	Workarounds, and documentation improvements.
1.1	2026-04-18	Tool additions, and documentation improvements.
1.0	2026-04-13	Initial release.

D.1. Changes in Version 1.5

- Added a subsection explaining why the dev container is slower, and more prone to corruption, on Windows, and recommending that confident students work directly under WSL instead.
- Clarified that Verilator warnings usually indicate real bugs and should always be eliminated, even on the rare occasion one does not.
- Explained why `enable` and `rst` are conventionally handled inside `always_ff` rather than the next-state logic, mirroring real flip-flop hardware.
- Made explicit that the project adopts the three-counter state encoding, so the corresponding oral-preparation question is answerable from the text.
- Various wording and stylistic polish throughout.

D.2. Changes in Version 1.4

- Added guidance on rebuilding the dev container without cache, for cases where the cached image is corrupted or incomplete.
- Suggested installing the `vscode-pdf` extension to view PDF files inside VS Code.
- Clarified the edit mode specification: fields below the selected field continue to tick normally; the selected field and all fields above it do not advance.

D.3. Changes in Version 1.3

- Added a pytest check that fails if any `rtl/*.sv` file has linting or compilation warnings or errors.
- Added `graphviz` to the installed packages, as required by the `yosys show` command to produce `png` files.

D.4. Changes in Version 1.2

- Added an explanation of SystemVerilog initialisation behaviour and updated the coding example accordingly.
- Clarified the use of width adjustments; in particular, that the original parameter should be passed when setting an instantiated module's parameter.
- Added installation guidance and common pitfalls.
- Added a workaround for the Verible extension failing to download the `verible-verilog-ls` binary on Mac.
- Fixed a missing flag in a code listing.

D.5. Changes in Version 1.1

- Clarified the project objectives.
- Added formal verification tools to the environment.
- Made minor changes to the environment configuration.
- Expanded the installation instructions.
- Added further explanation of the environment and its components.
- Added VS Code productivity tips.
- Documented additional common pitfalls.
- Fixed a typo (replaced `end` with `endgenerate`).